

IntentSpider - The Fluid Language Web for Textual Tension and Prediction

The Architecture, Valence Gated Extensions, a Gating System and a Evaluation Protocol with Debugging

Neksha DeSilva

Inverse Metafunction, Inc.

Research Paper



IntentSpider Webnet is dedicated to the memory of NASA's Apollo 11, and to the American heroes who, on July 20th, 1969, became the first conscious beings from the Planet Earth to set foot on another world.



Abstract

IntentSpider is an on device, privacy first, and privacy preserving predictive text engine, built and tested based on Standard Template Library in C++. This is a personal intent graph engine with dynamic edges in which every edge is typed by the accuracy of the typed words (or) array of characters without spaces, whether it reliably carries signal pattern towards what the users, in reality, go to select, rather than the time or how long it has been used. This decision was based on the analogy of the PageRank algorithm, used by Google Search Engine, seeded from several recent tokens, considered by how diversely each array's own reliable connections make more connections with other arrays (fanning algorithms), rather than with a direct search from a one token. The method is similar to how search engines rank webpages, it is an established technique, and we cover that part in detail. Those connections, which are labeled by their process, and the spread-weighted seed built on top of this, are novel subsystems, motivated by the weaver spider's web silk study.

IntentSpider core has one structural limitation, here we termed as the blindness to the valence, which implies that, it is unable to differentiate a selection or a next word that confirms what the user implied from the selection that closes the exchange with the other end(human, machine, or a processing interface) because neither the connection strength update nor updates on reliability considers the rest of the sentence that word came from. To address this, IntentSpider uses a signed, valence-gated extension, with a light, internal valence signal taken from WPM, or words per minute, via a simple algorithm and correction behavior based on previous research on typing errors and increased frontal theta activity in the human brain during keyboard interaction, rather than a default list of emotional words. This signal is also motivated by a finding from the orb spider's reflections. As an analogy, a web's

vibration frequency without a custom dictionary of words decides whether the IntentSpider approaches or withdraws. Even further, the separate gating mechanism, as declared here as necessity gating algorithm, built openly using a creative reconstruction of SCAMPER methods, to decide whether an array of characters should be added to the X number of the next words vector, which is independent of which word is chosen. Building on the valence signal, we add a second, independent signal, "hexarousal", which is a highly variable, intensity value, without a positive or negative direction, taken from how much a person's current typing speed (word count per minute) differs from their average speed.

In addition to this, IntentSpider includes six other engines, motivated by an attacking and struggle analogy of the Orb Spider, Wandering Spider, and the Rolling Isopod prey. This is a reinforcement that compounds across several live candidates at once. A radius of signals that grows under a consistent hexarousal and a sudden suppression event that returns a value for the shock that occurred. And it is separate from general, ordinary valence-based suppression algorithm. This stabilizing rule treats confirmed and unconfirmed excited states exchanges differently. IntentSpider also consists of an unsupervised discovery step that finds however many distinguishable excited states a person's typing actually shows per each user and a nonlinear combination of behavioral features in place of a single hexarousal number as well.

As a separate component, we add a graph theory based, live spectral embedding of the same graph per each user. turning each typed sentence into a path through a two dimensional working plane, and the delta between two sentences(a sentence implies an array of space separated characters) into a measurable object, as a shape rather than a single number. This feature does not change the system behavior and is not an input to the ranking process, reinforcement engine, or the gating process. This can be interpreted as a debugging system as well.

Keywords: ML, PageRank, Graph Diffusion, Entropy Weighted Seeding, Biometrics, Dynamic Graphs, Temporal Graphs Privacy, Predictive Text, Confidence Gating, Valence, Affect Models, Spectral Graphs, Human-Computer Interaction

Contents

1. Introduction
 - 1.1 Contributions
 - 1.2 Roadmap
2. Related Work
 - 2.1 Sequential and On-Device Predictive Text
 - 2.2 Complementary Learning Systems and Dual-Timescale Memory
 - 2.3 Consolidation via Parameter Protection
 - 2.4 Graph Diffusion and Personalized PageRank
 - 2.5 Local Computation for On-Device Feasibility
 - 2.6 Reinforcement and Inhibition Dynamics
 - 2.7 Signed Networks and Valence
 - 2.8 Affect Representation and Computer-Mediated Communication (*new*)
 - 2.9 Spectral Graph Embeddings (*new*)
 - 2.10 Positioning
3. System Formalization: A Function-Typed Personal Intent Graph

- 3.1 Setting and Scope
- 3.2 Nodes and Silk-Type Functional Differentiation (*revised*)
- 3.3 Role Assignment and Its Open Question (*revised*)
- 3.4 Architecture Diagram
- 4. Diffusion Mechanics
 - 4.1 Heat Kernel Formulation
 - 4.1.1 Fan-Dispersion Seed Construction (*new*)
 - 4.2 Personalized PageRank Formulation
 - 4.3 Local Push: Making Diffusion Feasible On-Device
 - 4.4 Ranking and the Entropy Signal
 - 4.5 **Geometric Trajectories: A Live, Two-Dimensional Embedding of the Personal Graph** (*new*)
 - 4.5.1 The Embedding: A Literal Two-Dimensional Web
 - 4.5.2 The Trajectory: A Sentence as a Path
 - 4.5.3 Heads and Plateaus
 - 4.5.4 Two Trajectories, One Plane: Shared Prefixes and the Gap Between Them
 - 4.5.5 Reconvergence and Its Relationship to Fan-Dispersion
 - 4.5.6 The Median Line (Open Question)
 - 4.5.7 The Inverse Problem: Retrodiction (Open Question)
 - 4.5.8 Relationship to the Existing Entropy/Confidence Signal
 - 4.5.9 A Worked Illustrative Example
 - 4.5.10 What Is Established Here and What Is Not
 - 4.5.11 Why This Stays Hedged
- 5. The Valence-Blindness Problem
 - 5.1 A Concrete Instance
 - 5.2 Why This Cannot Be Patched by the Existing Exhaustion Penalty
- 6. Proposed Extension: Valence-Gated Signed Diffusion
 - 6.1 Design Goal
 - 6.2 On-Device Valence Scoring via Typing Cadence (*revised*)
 - 6.3 Signed Edges
 - 6.4 A Third, Suppressive Set, and How It Now Relates to Transmission Fidelity
 - 6.5 Signed Diffusion Read-Out
 - 6.6 Algorithm
 - 6.7 An Open Theoretical Question: Does Local Push Still Work Here?
 - 6.8 Residue-Priority Arbitration via Logarithmic-Pool Recency/Context Interpolation
 - 6.9 **The Emotional Engine: Compounding Engagement, Growing Radius, and Shock-Retreat** (*new*)
 - 6.9.1 A Second Affective Axis: Arousal
 - 6.9.2 Compounding Cluster Reinforcement Under Sustained Engagement
 - 6.9.3 Growing Engagement Radius Under Sustained Arousal
 - 6.9.4 The Shock-Retreat Event
 - 6.9.5 The Settling Event (Grace-Period Timeout)
 - 6.9.6 Emergent Per-User Sub-State Clustering
 - 6.9.7 A Nonlinear, Multi-Dimensional Interaction Weight
 - 6.9.8 The Communication-Register Prior
 - 6.9.9 What Is Established Here and What Is Not

6.9.10 Why This Stays Hedged

7. **Necessity Gating: Deciding Whether to Suggest at All** (*new*)

7.1 Motivation

7.2 Generation Method: SCAMPER, Shown Openly

7.3 Proposed Mechanism (New, Untested)

7.4 Relationship to Residue-Priority Arbitration and the Full Pipeline

7.5 What Is Established Here and What Is Not

7.6 Why This Stays Hedged

8. Privacy Considerations

9. On-Device Feasibility: Order-of-Magnitude Estimates

9.1 Memory

9.2 Per-Keystroke Latency

9.3 Comparison to Existing Approaches

10. Proposed Evaluation Protocol

10.1 Data

10.2 Baselines

10.3 Metrics

10.4 Hypotheses and Pass/Fail Criteria

10.5 Ablations

11. Discussion: Novelty, Non-Obviousness, and Patentability

12. Limitations

13. Ethical Considerations

14. Conclusion

References

Appendix A: Correspondence to the Motivating Analogy

1. Introduction

Software keyboards such as modern Google Gboard, Microsoft Swiftkey, Apple QuickType, etc., predict the next word the user types from a local and sequential context by matching two, three or a "y" number of previous word units against a SLM, or a Statistical Language Model, and the most likely next words are suggested to the user.[14, 17]. If that array of words, in that order, was input to the system earlier. If the input engine did not register that particular combination or permutation of words earlier, its suggestions lose accuracy and predictive ability, since it relies on sequential processing, the system diverts to a separate, default dictionary system or an alternative method that offers generic predictions instead. It also identifies patterns stored similarly, regardless of how long it has been part of a human's stable vocabulary, compared to how recently and transiently it appeared. And it has no native process for differentiating a 'next word' that was later confirmed by the user's choice or behavior from the next word that was later rejected.

Production systems address this problem in Google's Gboard, for example, trains the underlying next word model using Federated Learning capabilities, so that information in raw forms will never transmitted but processed internally, and only combined and averaged model updates are shared to the server.[10].However, federated learning by itself does not resolve the empty start and context-dependent memory or Situational memory problems described there.

A federated language model uses local storage for sequential memory or its sequential context, And also, because it is trained on millions or billions of devices, it is, as obvious, not personalized for each individual device or the terminal node than an engine like IntentSpider, which operates entirely individually and builds only that human's own webnet of word connections, without any involvement of external servers.

IntentSpider is an on device, fully personalized alternative to those existing systems. It is a system that represents an individual's words and short, dynamic sentences as nodes of an intent graph, which is private and is not shared with any external party, and each edge is scored by the accuracy, meaning that how reliably it leads to the word the user actually ends up choosing. This score is a number that can rise and fall, rather than being fixed by how long the edge has existed so far or how often it has been used. And it is based on 'dispersion'. Then, IntentSpider sends this seed to the outward through only the edges that have earned a transmission reliable type status, using a personalized, PageRank type graph diffusion [16, 13, 1]. For that reason, first time word combinations or permutations can still generate a more accurate next word if it falls near an already well-connected, reliable region of the user's personal graph (motivated by the analogy of the vibration signal reaching the spider through most reliable threads) This component replaced an IntentSpider legacy version that instead handled two separate connectors or connector types which were operating with a difference in timescales. [19] [11, 21, 2, 22]

1.1 Contributions

1. It gives an accurate description of a diffusion architecture with assigned roles to them, In this architecture, edge's role in sending signals is assigned by the reliability of that edge so far, rather than using timers or clock function., And also, The novelty of the IntentSpider is compared with many other research and related work including the two timescale and elastic weight consolidation work (or EWC), which proposed in IntentSpider legacy version.
2. IntentSpider solves a structural problem in the IntentSpider legacy base architecture. This problem can be named as the blindness to the valency. And also, in this problem, neither the raw weight value update nor the reliability value update can separate a next word selection that confirms what the user meant from the next word that makes the text exchange with the other party final.
3. IntentSpider also gives a separate and independent process for filtering, which is "necessity gating" process for deciding whether a next word should be displayed or chosen. And also, this decision is separate of next word that the processes of Section 3 and Section 4 would rank first. And also, IntentSpider generates this necessity gating mechanism through a creative use of the SCAMPER method (Substitute, Combine, Adapt, Modify, Put as a one, Eliminate and Reverse, not Rearrange.), rather than from a new failure mode found in the system.
4. Our system shows a testable system with pass and fail return outputs, metrics and invented benchmarks (refer to Section 10), also, this process exists because of the reason that main claims of paper, that architecture of Section 3 through Section 6 outperforms a plain word prediction baseline (or n-gram methods), and also outperforms relevant systems that use two mem-

ory speeds to learn (to avoid forgetting), in both cases where lacking previous history to learn and mixed valance conflict cases., and that necessity gate reduces irrelevant suggestions without slowing the entire IntentSpider system or crashing the UI, etc. This experiment will be discussed even further in the paper.

5. There is a second system that operates separately like as the necessity gating architecture. Named as the 'affective axis', this can also be named as "hexarousal" alongside the valence signal.

In addition to that, IntentSpider has six more system processes (refer Section 6.9). These built entirely from an motivation by an attacking and struggle analogy of the Orb Spider, Wandering Spider, and the Rolling Isopod prey. This includes six systems. The first is reinforcement that accumulates in across several next words in live at the same time. Another is a diffusion radius value (motivated by the vibration ripple pattern) that makes the hexarousal move to higher levels when the diffusion radius value raises high. Another is rapid suppression (within a short time). Another is an event that executes the command right after the shock eventlister triggers. This event is separate from ordinary suppression. Another is a stability rule which allows the excited state a rerouting time before it turns to the ordinary state. The other step is a unique set of values, and it finds the x number of distinct excited states that human's current typing shows. And the last is a combination of behavioral features. Also, this is the most complex and the system which contains most parameters as well.

6. Also, this gives the same graph a 2D spectral embedding (refer to section 4.5). This means, it turns each typed sentence into a path through a plane, rather than by a sequence of abstract vectors.

In the IntentSpider Webnet, we created two testable points (or hypotheses) from this embedding. First, a node's connectivity breadth predicts its distance from the center of the structure. Second, the dividing sentences connect more tightly around a shared and narrow word than around a shared, broad word. Unlike other contributions, this hypothesis does not change the system's behavior. This can be considered as a debugging or an inspection tool built from outputs that IntentSpider already produce. And this is not a merged process to any other systems or processes.

2. Related Work to IntentSpider

2.1 The Sequential Processing Background of Current Keyboards

In text prediction systems, often, there are two types of techniques common in papers. The first one is classic n gram language modeling with smoothing. Katz back off model uses a shorter context whenever the longer one is missing and/or not accessible. [14] Kneser Ney smoothing [17] Kneser-Ney method directs the probability to contexts that are more useful, not the ones which are frequent in use. They are foundational to these systems because they have a small memory and cost. Andrew Hard, affiliated with Google Gboard, trained a recurrent next word model for a mobile keyboard using the Federated Averaging algorithm. In that system, the raw input stays within the device indefinitely. [10] It only sends language model updates, combined with many other devices,

that are transmitted to the server. This work partially motivated us to build the IntentSpider's privacy capabilities into a single, per-device basis structure that one device could compute entirely on its own, with no topological networking, such as without any involvement of a network. These are capabilities that Andrew Hard's Federated Averaging algorithm did not possess. Their model is limited by the sequential context limitation of the neural language model it trains. IntentSpider, conversely, is fundamentally different from that architecture as previously mentioned.

2.2 Usage of two memory systems or Dual Timescale Systems and the process of learning

The theory of two interconnected memory systems running at different speeds is a old concept. The concept did not start from a ML background. In the human brain, in cognitive neuroscience field, Complementary Learning Systems or CLS theories describes that the human hippocampus is showing these patterns:

1. fast learning speed compared to the neocortex
2. Sparse encoding or sparse neuronal representation, which means that neurons are activated in a nonlinear, way. (e.g.: playing a game) It activates the occipital lobe, Parietal Lobe, PFC and frontal lobe, in an unpredictable, sparse pattern. [11]
3. And changes by the incident type in a multidimensional way, as mentioned, it is not a predictable pattern. And it depends on billions of parameter that are dictated by the laws of physics. So it is individual and incident specific.

Conversely, the human neocortex operates in a learning memory method that is slow, statistical, and very distributed instead of being centric. In that way, a proper way to arrive at a conclusion is an electronic system that only learns fast are more likely to forget fast, and a system that only relies on slow learning are much prone to slow learning and in rare cases, memory overloads or SIGTERM errors. (C++ runtime error)

Professor Geoffrey Hinton and Dr. David Plaut [11] described "Connectionist networks" (referred as ANN or Artificial Neural Networks) where every connection holds and carries two weights. Each network in the Connectionist network consists of a slowly changing weight that stores long term knowledge, and a connected fast changing weight storing temporary knowledge and decaying back to zero on its own. Scientists showed that practicing with a small amount of associations is sufficient for the use. This lets the fast weights cancel out the interference the slow weights had built up. Also, (in Section 3.2), it replaces the split with a single function based linking set (or an edges set). And also, this research area is under active development currently. The specialty of this field is that it is being under sustained research and development since historically. Which matters for the novelty claim. Three recent systems show this same fast and slow pattern in different types of forms. This includes the DualNet, from Dr. Quang Pham at el's research, [21], DualNet splits the tasks into two separate learning processors, This includes a fast process that trains itself (without labeled data) and a slow process that that preserves the learned data and information using Elastic weight Consolidation. Complementary Learning Systems based Experience Replay (or CLS ER), manages two similar clones or copies of each network and then each copy updated at a variable number of speeds. Then CLS ER uses how much amount the two network copies do not differ on a exam-

ple run as a signal for which specific previous examples to replay. This implies that the system has a priority for the greater differences. [2], ITDMS (Refer Information Theoretic Dual Memory System), this system keeps two separate memory buffers separately which includes a fast memory buffer for new sample runs and a slow memory buffer for samples that has the long term value for runs, This choosing process is based on the information theory(ITMO methods as a subset of the Information theory) (refer [22])

These above 3 research papers were published between 2021 and 2025. Which were built on the CLS theory foundations. And all three use some version of the same fast and slow split an earlier version of this paper also used. Which means that, a novelty claim based only on that specific two channel structure methods will be weaker, compared to the already existing research.

2.3 Protecting System's already learned information

Elastic weight Consolidation or EWC supplied the mechanism an earlier version of this paper used for protecting an edge once it moved into a stable channel. (refer [15]) For each parameter, this EWC estimates that how much changing it would damage the performance levels on tasks the system already learned so far. This is an estimate that uses the theory "diagonal fisher information approximation". And this method makes changes to this in the future difficult and more complex. This specific resistance only increases. Once a parameter, or in the legacy IntentSpider version, an edge, is considered as "important". And therefore, EWC has no ability to reverse or change that previous state of importance. Refer to section 3.2, it states that, why this one way property is a disadvantage for the system. It should not be considered as a feature for a real-time, live and a personal graph (or per user graph). In such a graph, These types of connections should be able to remove their 'important' state with the same amount of frequency of instructions or the effort similar to how they obtained it. In this paper, EWC is only used in this paper as the source for baseline B3, Which is one of the comparison systems used in later sections (refer to Section 10.2). EWC also matters because Section 3.2's own "freely reversible" property is defined by contrast with it. In the IntentSpider system, this does not use EWC as a system mechanism.

2.4 Graph Diffusion System and Usage of a Personalized PageRank

Personalized PageRank builds on the original PageRank algorithm, which was developed by Larry Page and Sergey Brin [20], (more specifically its random walk model.) (refer [20]).In here, instead of restarting the process similarly across the entire graph, it restarts from one specific initial node, or a small set of initial nodes [13].Johannes Gasteiger et al [16] showed that this method, used on its own without the neural network's prediction step, improves how well nodes get sorted into groups when only some of them are labeled (which is known as the semi supervised node categorization in fundamental ML), also known as PPNP or APPNP. This gives one example of using diffusion by itself, rather than a trained classifier, as the last step in Section 4. This gives one example of a case where diffusion works without involvement with other system processes. It functions more than a trained categorizing engine as the last step described in the Section 4. In addition to that, there is a similar option for it which is called as the heat kernel formulation (refer section no. 4). Professor Fan Chung

[4] showed that if this process runs for the right amount of time, it turns into a variation of the PageRank algorithm previously discussed. Because of these reasons, the two constructions used previously in section 4 (refer Section 4.2 and section 4.3) are the same core idea. In that case, only difference between them is how long this diffusion process has been allowed to run in the system.

2.5 Background for Using Local Computation for Practical Applications

IntentSpider's use in PageRank variation random walk model, and computing an exact user based PageRank vector means solving a system of linear equations across the entire graph here. This requires a massive amount of processing power (the actual runtime time would depend on the specific device) if done repeatedly. This is not a capability a mobile phone can practically repeat on every single keystroke input Reid Andersen et al. solved a version of this problem previously in 2006 (refer [1]). Their local push algorithm computes an approximate personalized PageRank vector instead, by pushing mass outward from an initial node. Here, In this process, 'mass' is only added to nodes which still hold more than a small, fixed amount of unused value. And also, the entire process finalizes and ends in time which is proportional to $\frac{1}{\alpha \epsilon}$ and this time is not dependent on the number of nodes on the graph. This is the reason that in section 9, we claim that this system is possible and capable of running inside a mobile device like an iPhone. Without that, the diffusion step would need to process every word, word by word a user has typed so far, and also, for every single keystroke. Making it unrealistic.

2.6 Strengthening and Suppressing Edge Connections in IntentSpider (Graph Edges)

Ant Colony Optimization, or ACO, takes a massive space in terms of initial contributions to the IntentSpider development and research, which means that the edge strengthening and edge decay dynamics used for the raw weight system in Section 3.2, this methodology came directly from the Ant Colony Optimization (refer [5]). In this method, pheromone trails on a graph get reinforced by traffic (the analogy of ants moving through a field of clay), and these trails of the ants' foot fade according to the time passed so far when they are not used. And also, in addition to that point, After a pattern completes, the exhaustion penalty value temporarily blocks it from activating again. The after that, it remains restricted until that suppression period ends. This penalty system is derived directly from the research related to the Tabu mathematical search optimization stacking algorithm or commonly, "TS" (refer citation [9]), Which is a process that also blocks a the system from retaking the already and recently used value or the solution in general terms, for a fixed constant amount of time. In IntentSpider, this blocking system exists to stop the engine from choosing the exact same next word being chosen again and again. (refer to section 3.2, 5.2 and section 7)

2.7 Signed Graph Connections, and measuring The System's Emotional Tone Signal (or Valence)

In Section 6, it needs a programmable object for a type of graph. In this specific graph, some edges should actively push back against a spreading signal. And also, this push back is different from the general type of fail. Which means that

it is different from the failure to reinforce that signal. In the fundamental graph theory in math, this is not novel. Professor Dorwin Cartwright and Professor Frank Harary (refer [3]) turned Fritz Heider's theory, which is the way how society and its components, which means people manage to keep their attitudes and preferences in balance and consistent with the one of the great theories that used to build a structural balance theory. And also, they built this theory specifically for signed graphs. In addition to that, Jerome Kunegis and others (Refer [18]) proved a valuable information. The standard spectral graph theory methods is working with graphs with edges with the value less than 0.0 (zero). In addition to that, this method includes the the graph connection matrix (also known as the laplacian), spectral clustering, and diffusion kernels. And also, in addition to that point, after doing this, there are several implications that happen afterwards. In addition to that, the IntentSpider legacy version required our previous method to produce the valence signal using a lightweight and an already available source only. Which means that, typing speed and correction behavior can be given as examples for this lightweight signals. These signals can be derived from the device directly instead of using large computational needs for data. And also, this process needs to happen inside the device. That previous version used VADER (referenced in research as "Valence Aware Dictionary and Sentiment Reasoner"), which was a sentiment model based. (refer Dr. Clayton and Dr. Gilbert's VADER model [12]). In addition to that, VADER is not a neural network based model. It is a rule based model and the model VADER was built by the scientists to run it without data servers. Unlike the IntentSpider legacy version, in the upcoming section 6.2 now replaces VADER with a separate. This new signal is based on the specific pattern of the typing speed of the human, and also, it does not require a list of words which are created early. This new signal is motivated by the spider neuroethology study, which researches how a spider's nervous system produces responses (refer [26]). It is not motivated by VADER's research on reading emotional meaning from words that are being typed or input. And since the VADER research paper was a main source which was used to construct the theory, IntentSpider's this version also cites that specific paper. And also, VADER research is the reason why the section 12 can name sarcasm and indirect refusal handling as a specific limitation. This is a limitation the word list approach would have carried. In that kind of a case, VADER was built and tested for texts and lines found in social media posts, and text posts in general. (using data obtained by Twitter or now X). And it was not built or validated for this level of precise data harvesting, meaning that, keystroke level Intent Gating. Which implies that the accuracy of it in this more narrower study of IntentSpider was never tested.

2.8 The Reading of Affect and Emotional Tone in Text Communication Between Humans

Aligned with this, there are two already existing research papers that support the Affective framework which is being discussed in Section 6.9, and here, these two citations also support the warning of the calibration discussed later in this current section. After that, the first step is representing the Affect as a very small space included with independent axes, meaning that, axes that do not correlate with each other rather than as one single number. Professor James Russell identified and proposed representing (refer [28]) the affect in this

case, two independent dimensions, valence (pleasant axis to unpleasant axis) and arousal (activated axis to deactivated axis), arranged into a circular model, which means that circular model connecting the two dimensions. This specific research belongs to 1980. (refer section 6.9'1.) In addition to that, Professor Joseph Walther introduced the "Hyperpersonal Model" (refer [29]) which is an already exciting theory which names how online text communication is different to which specific parameters when compared to real life human chat. And this research establishes something very specific. Which means that the emotional nuances and the small obvious points in the communication is depending on the specific medium of the communication used. Which means, that difference changes differ with whether it is email, note taking, web messaging, or a Google search. Section 6 (specifically Section 6.9.8) is explaining this with factual basis. There is a typing rhythm based arousal or valence signal, hence, it is calibrated on a one medium, such as Instant text messaging (Skype, Signal). Due to that reason, this custom settings should not be equal to or substituted directly without editing to another medium's settings, such as Email or Twitter. Hence, this custom setting's value is left unspecified below. Due to that reason, it is not assigned a value it has not earned. But not any of these citations support the other six mechanisms in the section 6,9. Six of them are, compounding reinforcement, growing radius, shock retreat, settling, sub state clustering, and the nonlinear interaction weight. These six mechanisms can be considered as invented mechanisms.

2.9 Mapping and Placing Graph Connections Into a Smaller Space (Related to Spectral Graph Theory)

In the Section 4 of the IntentSpider paper, the Section 4.5 is an already existing idea. Which demonstrates that from the eigenvectors of its Laplacian, a graph's structure can be recovered in a low dimensional form. In addition to that, Professor Miroslav Fiedler [30] researched and found that The eigenvector tied to a graph laplacian's second smallest value of eigenvalue gives a one number as a numeric for how much is the interconnectedness of the entire graph. Which means that, a higher value implies that the graph is more challenging to disconnect into different sub parts. Which is now known by researchers as "the Fiedler vector". Because of this one result, this specific technique makes this possible to use Laplacian eigenvectors for separating a graph or connecting into it. Professor Mikhail Belkin and Professor Partha Niyogi [31] built the specific technique Section 4.5.1 uses directly. They represent each node by its coordinates in the eigenspace of the top few non trivial Laplacian eigenvectors. This produces a low dimensional embedding that keeps local graph structure in place. It is related to, but distinct from, PCA or classical multidimensional scaling. Section 4.5 applies this established technique to a live, per user, keystroke evolving personal intent graph. From this, it pulls two testable geometric predictions about connectivity and the reconvergence. As said, from those already established researches, non of those researches creates or implies those specific hypotheses used in the IntentSpider system.

2.10 The Novelty and Claims of IntentSpider Webnet

After reading from section 2,2 to 2.9, One of the conclusions that we will be able to take is that, the text prediction and more specifically, the on device text

prediction is a established area of research in human computer interaction. In addition to that, the theory of Two timescale memory architecture is an already existing area of research study with about minimum of three research architectures published from 2020 onward until 2025. Those architectures are also using the CLS (or Complimentary learning systems theory, methods) architecture used in the base of the IntentSpider as well. In addition to that, protecting learned parameters, diffusion based classification, local computation inside the device, reinforcement and dynamics of inhibition , signed graph methods, keystroke dynamics biometrics, and spectral graph embedding are also previously researched concepts into a some extent. And also according to several biological findings related to Orb weavers and the research related to it support these ideas as well. Which means orb weaver silk comes in separate types, each type of silk is for a one specific function. Tiger wandering spider's (scientific name: *Cupiennius salei*) frequency dependent approaches, and its withdrawl response and Professor Russell's circumplex model related to the valence and arousal.

We have not found a system that types graph edges by an ongoing, freely reversible measure of whether they actually predict what a user goes on to select. This is different from typing edges by elapsed time, usage frequency, or a one way consolidation clock. And also, we have not found a system that reads that typed graph out through diffusion, seeded by a combination of several recent words weighted by how scattered their connections are, rather than by the single most recent word. Also, we have not found one that adds a signed, valence gated extension whose valence signal comes from behavioral timing rather than word content. That is some of the claims that IntentSpider Webnet paper makes. And the research behind the claim above was not repeated specifically against the affective-computing and text-based computer-mediated-communication (CMC) research of Section 2.8, for compounding reinforcement, growing engagement radius, shock-retreat, grace-period settling, or emergent sub-state clustering. In section 11.2, it shows that this component's contribution to the paper's overall novelty case separately, and as clearly weaker than the four components the claim above actually covers. Nor does the claim extend to Section 4.5's geometric trajectories, for a different reason. This is not a claim of unverified novelty. It is a claim of a different kind entirely. The claim above is about a predictive architecture, what gets suggested and how. Section 4.5 predicts nothing. It observes and measures what the existing architecture already produces. Whether representing a live, per user diffusion process as a two dimensional keystroke path has been done before, in exactly this form, was not the subject of a dedicated search for this revision. Section 11.2 treats this component on its own terms, as a candidate way to look inside the system, not as a candidate improvement to prediction quality.

3. A Novel, Personal Intent Graph with Function Typed Edges

3.1 *Where and How This System Runs*

IntentSpider system is operating fully inside a single device. And for a single user. With IntentSpider, the server side components, external servers to transceivers are absent. And also, there is no combining of data across multi-

ple different users. This is not only just a preference but a self imposed constraint in design. This structure of IntentSpider makes the privacy arguments and concern in Section 8 minimal and well regulated, even without the need of a cryptographic or differential privacy proof (examples for DP include Apple Quicktype, Apple IOS and examples for cryptographic privacy proofs include Signal messenger, Coinbase's wallet). Which implies that the graph described below is built from only that specific human's personal typing history. And it is as mentioned, not transmitted to any other separate server via any medium. Those data and/or information are not synced internally in a readable form as well also, those data are not used to update or improve or accumulate into any other larger Webnet or a network.

3.2 Nodes, The impact on function based on the specific thread (or silk) type

The Motivation

Orb weaver spider webs are built from structurally and also physically distinct silk types. And each of those types of thread is for a distinct and a specific purpose. Radial threads and frame threads are created using a thick and a dry silk. Those thread types support the geometric structure of the spider web. And also these threads transfer and transmit the vibration signals again, back to the spider that produced them. But this spider web's spiral, which is used to catch the prey, has a very low thickness and a high flexibility, and also the ability to hold (or the sticky described in Dr Foelix's paper as "Sticky"). These are the type of spider silk that is also strengthened by the chemical gland of the orb weaver spider, which produces the silk. Which means that the purpose of this type of thread is to stably hold and keep specific data objects. Which means that, the specific thread's purpose or the role is not decided by the time it has existed, but instead, it is by the specific physical properties of that thread. This motivates replacing a time based channel split with a function based one. An edge's role in the graph is determined by whether it reliably carries signal to a useful place, not by how much accumulated support value it has built up over time. Let V be the set of tokens, meaning words or short word groups, that this specific user has typed. This set grows as new tokens appear. Rather than two weight functions across two separate edge sets, IntentSpider now keeps a single edge set " E " on " V ". Every edge, referred to as u and v , in this set carries two values, and each of those values independently updates itself. Raw weight. In each of those selection events, meaning the user accepts a suggested continuation v following context u , the raw weight is reinforced, (or strengthened). In other cases, it starts to decay in exponential value, according to the time since it was last used. This works in the same way as the pheromone trails of the ants inside the Ant Colony Optimization algorithm (or ACO algorithm) (refer to [5]).

$$w(u, v, t + \Delta t) = w(u, v, t) \cdot e^{-\Delta t / \tau_v} + \eta \cdot \mathbb{I}[\text{selection event at } t + \Delta t] \quad (1)$$

In this context ' τ_v ' represents a short sized decaying constant (referred as Tau sub V constant) which is programmed to be active between several hours and several days. This specific speed of the decaying correlates and decides how quickly the strength of a word connection lowers or decays when a user does not choose it for a specific period of time. And also, this method makes it sustainable by programming the tokens so the per user graph remains dynamic

by force deleting earlier, old associations to allocate space for new patterns. This specific symbol ' η ' refers to the relevant learning rate used by this process. This learning rate denotes and correlates with the the amount of the weight of a specific connection increases each single time when a human selects a word. Which means, in this method, this successfully controls the speed with which the engine adapts to new typing patterns and learns the human person's personal preferences.

In addition to that, this above equation (1) updates one single edge per each selection event, Which means, based on the specific moment when a human accepts a suggestion, below in Section 6.9.2, this paper introduces an additional word group strengthening term. This term layers on top of the existing update, rather than replacing it, and it only applies when several candidates are continuously chosen by user and also while live at the same time. This specific new reinforcement component uses its own initial reinforcement value. And in here, there is a specific separate symbol for this value as well in order to avoid the confusion points with η in other calculations.

To manage the overload ("exhaustion" in this equation) our system uses Tabu Search algorithm which was developed by Prof. Glover and which is one of the most core and important features as it was in the legacy IntentSpider version. This specific method with Tabu search prevents the system from suggesting the same word too many times. And as the section 2.6 demonstrates, After a specific selection event occurs by a human, the system blocks that edge for a variable amount of time. And this specific blocking process (in this case, this blocking process and reinforcement is separate and not correlating each other.) is a separate process.

$$\text{exhaustion}(u, v, t) = \exp \left(-\frac{t - t_{\text{last}}(u, v)}{\tau_x} \right) \quad (14)$$

In this Specific case, ' $t_{\text{last}}(u, v)$ ' represents the time of the most recent selection event on edge (u, v) , and τ_x represents a short suppression period, which Section 2.6 explains in more detail (refer [9]). This τ_x symbol is kept as a separate notation than the " τ_v " notation which correlates and decides in which kind of speed the raw weight decays in that Equation (1), and also separate from the recency constant τ_0 , (which is the constant that is introduced in the section 6.8), where it paces the arbitration support decay instead. $\text{exhaustion}(u, v, t)$ is a separate accounting signal by itself. (As a note, this specific signal remains closer to the 1 that specific moment after the selection. and as the time spends, that specific value starts to decay down to 0). It is not subtracted from $w(u, v)$ in Equation (1). And the Section 7 uses this signal downstream. In section 7, there is a signal in order to decide whether a candidate should be shown but it is not used in order to change any already stored edge weight.

This section introduces and defines transmission fidelity. in this theory, every edge object includes and has assigned to a transmission fidelity value, which is written here short as "TF", and this specific value is built with u, v and t values and the TF value if always between 0 and 1. (As a note, this specific TF value only updates itself at that moment the connection sends a specific amount of diffusion mass toward a next word the human selected after that event.)

$$TF(u, v, t + \Delta t) = TF(u, v, t) + \eta_{TF} \cdot (\text{outcome}(u, v, t) - TF(u, v, t)) \quad (16)$$

3.3 Edge Role Assignment Process and Theories

$$(u, v) \text{ is } \begin{cases} \text{transmission type} & \text{if } TF(u, v, t) > \theta_{TF} \\ \text{capture type} & \text{otherwise} \end{cases}$$

Equation explanation - In here, the transmission type edge is included in the diffusion matrix $W_{\text{transmission}}$ and enables the diffusion process. The capture type edge is excluded from the matrix and is added and placed only as a direct and a non forwarding association.

The EWC's importance value which is referred as 'F' takes in u value and v value combined and afterwards, this value is only increasing the equation 2 (Eq. (2)) which was in the IntentSpider legacy version. This specific process resulted in the edge that was complex, limited and reduced the ability of the capacity of the edges to change. But the TF val, (transmission fidelity value) and then processing value "v", "u" and "t" as a single array of data (by combining) can move in either of those directions as the human's typing patterns change. And also, an edge that fails to provide the value to the system results in a reduction from its transmission status. Which means that non of those edges are permanent in this system. After that, The legacy IntentSpider version was not capable of unlearning the already learned but falsely stable and also outdated associations and information due to a specific lack of a process for automatic and more natural cleaning. This is also discussed in section 2.3 and equation no. (2) and no. (3). To recorrect this issue, equation (16) and equation (17) invents a new process for unlearning.

This section is about the reliability of above processes. A new edge with a consistent log success history is converted into a type of transmission after a brief of time. In that same way, But also, an edge that is being used frequently but has very low (or zero) confirmed selections cannot be converted into a transmission type regardless of the specific timeframe or the age of the edge. In addition to that, the legacy IntentSpider version tracked the recency and the reliability as a combined object on a single promotion clock, This IntentSpider version separates them into separate quantities.

This section uses η_{TF} as the main free parameter for this process. This processing system does not require a fixed window "T" since "TF" calculates an ongoing exponential moving average instead of a specific windowed count. And also, compared to the legacy version of IntentSpider (means, before the revision), this system reduces the number of free parameters by one

3.4 An Overview of the Architecture in Figure 1

In here, a new input for example, such as a pizza order, accumulates a raw weight and after that begins obtaining a transmission fidelity value (alt. referred as the TF Score). An edge whose transmission fidelity passes θ_{TF} becomes a "transmission type" and added to the process diffusion (refer section 4). An edge that does not pass this specific margin value is a "capture type" and then remains available only as a direct, local, immediate and non forwarding association. This architecture is different from the previous architecture with two channels. And unlike in legacy version, in this architecture, movement between these two roles is continuous and reversible in both directions as $TF(u, v, t)$ value changes.

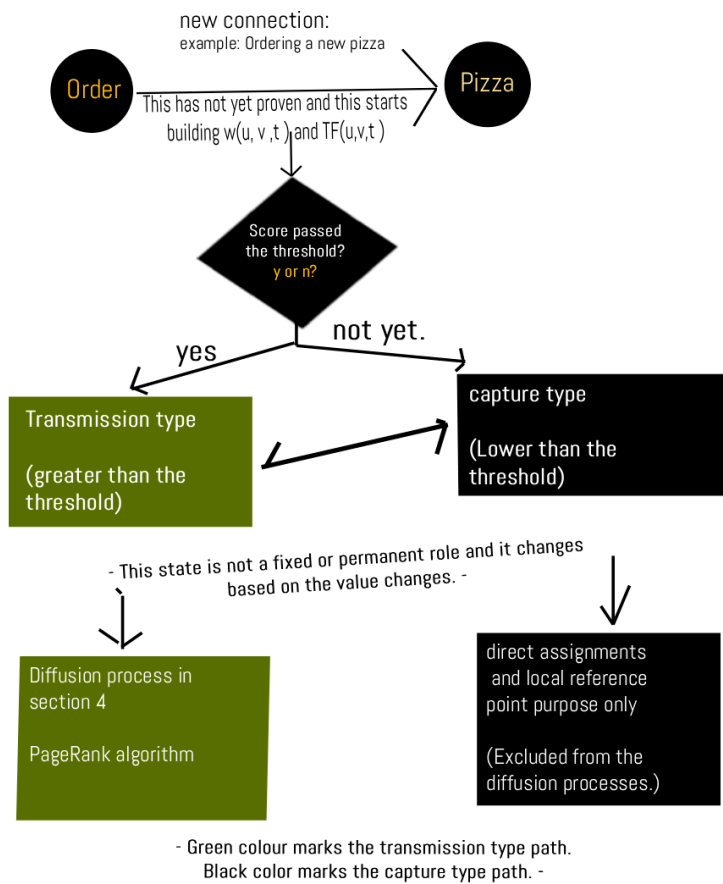


Figure 1. Figure 1 shows the single, functional and function typed edge set and the two possible states for the edge.

4. The Process of Diffusion

The diffusion step decides how a newly typed word, or several of those words, produces the ranked suggestions. In this process, transmission type edges ($W_{transmission}$, refer to Section 3.3) are used only for propagation. And also, capture type edges absorb new input, but those capture type edges are intentionally excluded from the long range diffusion. Meaning a transient, situational, or unreliable pattern cannot flood the suggestions in the same process a reliably predictive one can. As a note, this paragraph, and also the heat kernel and the personalized PageRank equations below, are unchanged from the earlier two channel version of IntentSpider Legacy version.

4.1 Heat Kernel Formulation

Let $L = D - W_{transmission}$ be the graph Laplacian of the transmission type edge set, and in this case, D is its degree matrix. The heat kernel at the diffusion time t is

$$H_t = \exp(-tL), \quad s(t) = H_t \cdot s(0) \quad (4)$$

In this scenario, $s(0)$ is a seed vector, which is constructed as described in Section 4.1.1 right below it, rather than as a 0 or 1 value indicator on the single currently active word. Which means that, as "t" value increases, the signal is shared via using the transmission type edges. And also, nodes which are strongly connected and nearby receive relatively more signal than the ones in the further distance. Professor Chung (refer [4]) noted the precise correspondence between this continuous time process and PageRank algorithm. And this specific correspondence is the reason which licenses treating the two formulations in this section as can be substitutable for one and the another.

4.1.1 Creating Seeds from the Node Dispersion Algorithm and Using Shannon Entropy in the Information Theory to Rank Previously Typed Words by the Human

A seed vector with a single 1 at the just typed word is considering every word as a equal and a similar origin or a starting point for the diffusion processes. Regardless of the specific state of the connection or the connectivity of that specific word, this process executes. Meaning that, this method does not consider and it actively does not include whether the usefulness of the specific signal that it carry. This also does not include whether the word connects to many other words that the combined and collective signal is not clear. In addition to that, in English, connecting words like 'the' or 'in,' and with other completing or filling sentences such as 'you know', 'of course', 'by the way' are soem specific examples of this problem. Which implies these objects (sentences or words can be considered in a high probability and less variance, as they can be used before or after many types of words.). Due to that reason, if one of these words is used by itself to begin this process, the signal reaches every part of the entire graph in equal amounts. And it is not moving toward the human's underlying intent.

For node u , let $\pi(u, \cdot)$ be the its relative outgoing weight distribution, limited to the $W_{transmission}$ edges. This limit is the specific point of integration with the previous subsection no. 3.2. Meaning that, in here, the system only measures the dispersion values for the edges which are already in the state "transmission type" Hence, a relevant node's dispersion value is showing the correlating

shape of its reliable connectivity. And it does not showcase the raw type connectivity(without shape) .

If we define

$$\varphi(u) = \frac{H(\pi(u, \cdot))}{\log(\deg_{\text{transmission}}(u))} \in [0, 1] \quad (18)$$

This uses the "Shannon Entropy " which is a basic theory discussed in the Information Theory. And before this, we introduced Shannon entropy $H(\cdot)$. It is also for the ranking confidence signal in the section 4.4 below. And for a second role, it is now again reused. Means that, this does not evaluate the confidence values of the output distribution but rather, it is the dispersion of the specific input node's internal local connections profile. If a word's transmission edges spread evenly across many other words, $\varphi(u)$ equals a value which is close to 1 approx. If a word's transmission edges focus on a few amounts of next words, $\varphi(u)$ equals the value close to approx. 0.

The seed vector for the last "k" typed words $u_t, u_{t-1}, \dots, u_{t-k+1}$ is then

$$s(0) = \text{normalize} \left(\sum_i (1 - \varphi(u_{t-i+1})) \cdot e^{-(i-1)/\tau_{\text{seed}}} \cdot e_{u_{t-i+1}} \right) \quad (19)$$

τ_{seed} does not share a symbol with any wall clock decay constant in any other place as well. Those are τ_v , τ_x , and τ_0 . And the subsection 6.8.3 is similar for ρ and γ , for that similar reason. τ_{seed} is reducing the word's specific role or the part in the seed as that word's position in the typed sequence moves further back. But it does not reduce based on elapsed real time. A word with no transmission type edges has an undefined $\varphi(u)$ value. And this word does not need a special case in Eq. (19). Which means that, this kind of a word cannot send signal. Due to that reason, it does not matter what seed weight the equation gives to this word. And the equation naturally gives this word no effect.

Whether the words that are directly next to each other should be combined into a single compound node before Equation (18) and Equation (19) process them is not solved by the theories and processes described above. Which means this is a question that is open and available. Meaning that a two word group such as "you know" or "I mean" or "in fact" or "for sure" or "no way" would output one dispersion value, and not two separate values. In addition to that, an extension that is not tested would use low entropy at the Bigram transition level ($u \rightarrow v$ used every time) as the method for combining u, v into one node. Means it uses the entropy tool for a third distinct purpose (wordization). This is separate from the ranking confidence and the seed weighting. Also this can be considered as an unverified addition to the system. And this is not relented to or derived from Equation (18) and Equation (19). This part is also listed in the section 12 as well.

4.2 Personalized PageRank Formulation

The practical, on device version replaces the matrix exponential with a fixed point equation. Let $P = D_{\text{transmission}}^{-1} \cdot W_{\text{transmission}}$ be the row normalized transition matrix of the transmission type edge set. The personalized PageRank vector p satisfies

$$p = \alpha e_0 + (1 - \alpha) P^T p \quad (5)$$

In this context, e_0 is now the fan dispersion weighted seed of Equation (19). And it is not a indicator vector on a single input word. This is the same change done to $s(0)$ above. And this is mentioned here again because of a specific reason. Equation (5)'s e_i and Equation (4)'s $s(0)$ are the same seed. Professor Fan Chung (refer [4]) showed that those two are the same seed at different points on a diffusion time axis. And also, both of them need to update together. If one of them is not updated, the other should not be updated either. In addition to that, $\alpha \in (0, 1)$ still controls distance this process pushes the signal before it restarts back to the seed. And this is the method also known as PPNP or APPNP (refer [16]), and, in general terms, the personalized web search (refer [13]) as well.

In subsection 6.9.3 below, The IntentSpider system considers α as a fixed constant value which is visible in above as well. In addition to that, section 6.9.3 proposes a different process. In this specific process, an effective restart parameter lowers on each keystroke, as a saturating function of the timeframe and for what amount of time the a user's typing remained in a sustained state in arousal. Meaning that, this process increases or extends the signal more further when it is a sustained engagement. Which means that, the walk runs more longer initially before the state restarts back to the seed state. And this uses a separate symbol (α_{base} , $\alpha_{effective}$) that does not replace the α used here. (and should not be confused) In addition to that,

4.3 Local Push: Making Diffusion Feasible On-Device

Solving this Equation 5 requires inverting an $|V| \times |V|$ system. Which means that, it is not feasible to repeat on every single keystroke. This is because $|V|$ can increase into over 10,000 or even higher amounts of words for a daily computer user over the years. In addition to that, Professor Reid Andersen and others (refer [1]) give an algorithm for this. This algorithm is named as "the local push." In this method, it sets an approximate solution $\hat{p}$ and a residual vector r instead. And after that, it repeatedly pushes mass from any node which has a residual value greater than ϵ to its nearby neighbor values. And it ends once no residual is over the margin value. Their central result is a runtime bound of

$$O\left(\frac{1}{\alpha\epsilon}\right) \quad (6)$$

And this bound is separate from $|V|$. Which means that, it depends only on distance the signal actually needs to travel. And also, on how precisely it needs to be resolved. This is the property this architecture depends on for the on device feasibility. This is quantified further in Section 9. Which means that, the cost scales with the shape of the local neighborhood around the just typed word or words. And it does not scale with the size of the user's entire typing history.

Section 6.9.3 below adds a third point. And this is a separate type than the two above. The signed weights question and the question about the edge set changing on each keystroke are about whether local push finalizes and ends in a fixed amount of time. Which means, whether local push finalizes and ends once the graph it runs on has values which are not negative values, and once the edges in the transmission type edge set are not the same on each keystroke.

In addition to that, the growing radius mechanism is different from these two. It does not decide whether local push finalizes and ends in a fixed time for the " α " used here. Instead, the disadvantage is, this is a process which lowers α value on each keystroke, as the user's typing remains in a sustained engagement. And also, this disadvantage increases when α lowers. it is not decided by whether local push finalizes and ends, but instead, it is decided by the " α " value this process includes. where the " α " is a value that reduces by itself. Due to that reason, this system discusses it as a separate one. And it does not combine it into the same open question as the other two. (refer section 6.9.3)

4.4 Ranking and the Entropy Signal

4.5 Ranking and the Entropy Signal

The next words are ranked by the relevant resulting values in p (or $s(t)$). And they are not ranked by conditions such as the specific proximity (like as directly next to the input) There are two results and implications of this process.

- A word with no direct co occurrence history with the input can still be a high ranked suggestion. This state is viable when that specific word is in a interconnected region area of the transmission type graph which the relevant seed reaches. And this specific scenario where the system is starting from a lower amount of data (or cold start state) contributes to the motivation of this process.
- The previously stated Shannon entropy of the resulting distribution $H(p) = -\sum_v p_v \log p_v$, is a usable confidence signal. Which means, a p which is focused on a one intent depicts the intent values are more clear and a p which is a high in entropy depicts the intent is not clear. In that case, the system process use it to decide whether to show one single suggestion, or several separate next words. This is the first of the entropy signal's three now distinct uses in the IntentSpider system. The second point, $\varphi(u)$ in Equation no. (18), measures an input word's own fan out values and not the specific shape of an output distribution. The third point is not a verified point so far as of now. It is in Section 4.1.1's open question section. All three of those are the similar mathematical object which is applied to three different distribution processes. Due to the nature of the confusion, they should not be confused as well.

Section 4.5.8 below introduces a geometric option for the same confidence question that this specific entropy signal already answers. Meaning that distance from the embedding's centroid, and not the specific shape of the relevant distribution. And it is built from separate tool in mathematics which is the Laplacian eigenvectors. (not to be confused with the entropy). Hence, it is not a fourth use of $H(\cdot)$. And also, it does not add a fourth point to the three above. In addition to that, Section 4.5.8 denoted that those two signals are not similar and identical, and one signal should not be used alternatively or substituted for the other one. And these two are not shown here to correlate with each other. Both signals can be in relationship to the stability of the output of the diffusion process as well. This stability will be tested later in the paper.

4.6 Geometric Trajectories: A Live, Two-Dimensional Embedding of the Personal Graph

4.6.1 The Embedding: A Literal Two-Dimensional Web

Let $L = D - W_{\text{transmission}}$ be the same graph Laplacian which is already defined in the section 4.1. From the eigenvectors of its Laplacian, a graph's structure can be recovered in a low dimensional form. Professor Miroslav Fiedler (refer [30]) researched and found this first. And that research is for the single eigenvector which is tied to a graph laplacian's lowest eigenvalue which is not a zero value. (this is also known as the algebraic connectivity) In addition to that, Professor Mikhail Belkin and Professor Partha Niyogi (refer [31]) built this idea further into a usable low dimensional embedding. They did this process by solving the generalized eigenproblem $L f = \lambda D f$ for the eigenvectors ψ_1, ψ_2 which are tied to the two smallest eigenvalues $\lambda_1 \leq \lambda_2$ which are not zero values. And in here, the less complex or more trivial and constant eigenvector at the eigenvalue 0 is intentionally excluded. And after that, this method gives each node u the fixed coordinate

$$(x_u, y_u) = (\psi_1(u), \psi_2(u)) \quad (37)$$

And this coordinate is computed again as the graph grows when new tokens appear. A personal typing graph can be a graph which is not connected. Which means, separate and unrelated clusters of the vocabulary, which a user never types near one another, is a thing that can happen. And even a likely one. And also, the Equation (37) considers this graph as a connected graph. Which means that, a graph which is not connected has a zero eigenvalue with the multiplicity which is equal to its number of connected components. And it is not one. Hence, this changes which eigenvectors are considered as the two smallest values which are not zero values. This is the specific part which makes the project's own "hexagon with the prey in the corners" spatial picture true in a literal way for a one specific claim. And it is not only an analogy. And this picture is already the basis of the motivation in the Appendix A's first row. A node which its transmission edges spread evenly has a high $\phi(u)$ value. (refer Equation no. (18)) Which means, that specific node is pulled by many nearby neighbor values in many directions. Hence, it should remain closer to the embedding's centroid. And also, a node which its transmission edges focus on a few amounts of next words has a low $\phi(u)$ value. Which means, this node is pulled in one direction by a few strong edges. Hence, it should remain in a separate region area. Which means, the $\phi(u)$ value should correlate with $\|(x_u, y_u) - \text{centroid}\|$.

4.6.2 The Trajectory: A Sentence as a Path

$$\text{position}(i) = \sum_v \hat{p}_i(v) \cdot (x_v, y_v) \quad (38)$$

A sentence's trajectory $T = (\text{position}(1), \text{position}(2), \dots, \text{position}(n))$ is a path through a plane. And it is a one point per each keystroke. Which means that, this is a curve through the plane, and it is not only an analogy.

4.6.3 Heads and Plateaus

Define the trajectory's speed at the step i

$$\text{speed}(i) = \|\text{position}(i) - \text{position}(i-1)\| \quad (39)$$

A head is a local peak in the speed(i) value which is over a prominence margin value θ_{head} . Which means, the trajectory moves quickly toward a specific region

$$head(i) = \begin{cases} 1 & \text{if speed}(i) \text{ is a local maximum with } speed(i) - \max(speed(i-1), speed(i+1)) \\ 0 & \text{otherwise} \end{cases} \quad (40)$$

This head is expected to correlate with a word with a low $\varphi(u)$ value entering the recent seed window. Which means that, this is the section 4.5.1's hypothesis, which is applied here to a single step. And it is not applied to the entire graph. In addition to that, a plateau is a run of at least $n_{plateau}$ steps which happen one after the other. In this run, the speed(i) value remains below a tolerance $\epsilon_{plateau}$

$$plateau(i) = \begin{cases} 1 & \text{if speed}(j) < \epsilon_{plateau} \text{ for all } j \in \{i, i-1, \dots, i-n_{plateau}+1\} \\ 0 & \text{otherwise} \end{cases} \quad (41)$$

And also, the plateaus are expected during runs of words with a high $\varphi(u)$ value. Or when a trajectory already remains near a well connected cluster, and the further words only confirm it. This can give the heads and plateaus picture a precise and a good and a testable definition.

4.6.4 Two Trajectories, One Plane: Shared Prefixes and the Gap Between Them

If two sentences A and B share the exact same word prefix of length k, their trajectories match exactly over that prefix

$$position_A(i) = position_B(i) \quad \text{for all } i \leq k \quad (42)$$

This is because the section 6.9.3 introduced the $\alpha_{effective}(t)$ value in Equation (29). And this is a parameter inside the diffusion process. It depends on the $\ell(t)$ value. And this value is for what amount of time the user's typing remained in a sustained state. Which means that, it is a function of the typing rate, and it is not a function of the word content. Meaning that, two sentences with the exact same word prefix, which are typed at separate speeds or in separate arousal states, can carry separate $\alpha_{effective}(t)$ values through that shared prefix. Hence, a different $p^{\wedge}_i$. And hence, a different position(i) value. And this happens even though the words match exactly. In addition to that, Equation (42) is still correct as a statement about two possible continuations which branch from one real and already typed prefix. Meaning that, both continuations receive the exact same state until the branch point. (the graph, the arousal baseline values, and the sustained state length) But for two separate sentences which are typed on two separate times, this needs the stronger meaning of the "exact same word prefix".

Once the sentences diverge, define the gap at the step i as a 2D vector

$$gap(i) = position_A(i) - position_B(i) \quad (43)$$

The sequence of these gap values creates a region with an area. And it is not a single divergence number. And this area can be computed directly via using the "shoelace formula" (new) over the closed polygon which is made by

both paths and their shared starting point. This is because a shape which opens and closes carries information which a single number does not include. For an example, this shows in which way the two trajectories diverge. (a wide and an early gap, compared to a narrow and a late gap which reaches the same final distance) And it is not only the final distance value.

4.6.5 Reconvergence and Its Relationship to Fan-Dispersion

When both sentences later share a word again, whether the $\text{gap}(i)$ value closes back toward zero depends on that shared word's own $\varphi(u)$ value. And this is the most specific claim in this section. This is because it connects two previously separate parts of this paper into a one geometric explanation of the same underlying idea. Those two parts are the spatial embedding in the section 4.5.1 and the fan dispersion weighting in the section 4.1.1:

- A shared word with a high $\varphi(u)$ value has a broad and a generic connectivity. Which means that, it pulls both trajectories in a weak amount, and only toward the embedding's centroid. Hence, this is a weak and a generic closeness. And it is not a real reconvergence.
- A shared word with a low $\varphi(u)$ value has a narrow and a specific connectivity. Which means that, it pulls both trajectories strongly toward the same distinct cluster. Hence, this is a real and a valuable reconvergence.

This is also similar to the points made in the section 4.5.1 makes about single trajectories theories. And it is applied now to two trajectories. Which means that, words with a low $\varphi(u)$ value hold and keep the position strongly. Hence, a word with a low $\varphi(u)$ value, which is shared by two diverged trajectories, holds both of them in the same place. And it closes the gap. And a word with a high $\varphi(u)$ value holds the position in a weak amount. Due to that reason, sharing this type of a word closes the gap only in a small amount. Meaning that, only in the amount any two separate points which are added toward the same centroid can close.

4.6.6 Relationship to the Existing Entropy/Confidence Signal

The section 4.4 already gives this architecture a confidence signal. This is $H(p)$, the entropy of the diffusion process's output. And it is used directly in the necessity score in the section 4.5.1. A word with a low entropy value. (which is a high confidence in the section 4.4) And it is expected to remain further from the centroid. (which is the section 4.5.1's hypothesis) And this is the common scenario. And in this scenario, the two signals do not differ from each other. But a p which is spread evenly between two nodes in opposite directions.

4.6.7 A Worked Illustrative Example

Sentence A: "I want to buy pizza ... games." And also, Sentence B: "I want to make coffee ... games."

Step	word(s)	$position_A(i)$	$position_B(i)$	$\ gap(i)\ $
1-3	"I want to"	(0.15, 0.10)	(0.15, 0.10)	0 (exact, Eq. (42))
4	"buy" / "make"	(2.0, 1.0)	(-1.5, 1.8)	3.59
5	"pizza" / "coffee"	(2.3, 1.3)	(-1.8, 2.4)	4.25

Step	word(s)	$position_A(i)$	$position_B(i)$	$\ gap(i)\ $
6	"games" (shared, low- φ)	(0.6, -1.9)	(0.4, -2.1)	0.28

The gap value increases at step 4. Means there is a head for both trajectories (refer Eq. (40)). And the two trajectories separate into distinct and separate clusters. And after that, this gap value increases more at step 5. And after that, this value reduces in a large amount at step 6. In this step, a shared word with a low $\varphi(u)$ value pulls both trajectories to the same region area. Which means, this is the reconvergence. And it is directly visible as a shape which is closing up. (refer Section 4.5.5) And it is not computed from a one number in the way the earlier version of this paper would have listed it.

4.6.8 What Is Established Here and What Is Not

Component	Status
Theories and Processes named as Spectral embedding via Laplacian eigenvectors, centroid, Euclidean distance, shoelace formula area (Section 4.5.1, Equation no. (37))	Established mathematics [30, 31], applied here to a new, live, per user object
$\varphi(u)$ predicts distance from centroid (Section 4.5.1)	New, proposed for status, see the validation and experimentation section described later in this paper (refer Hypothesis 7)
Centroid trajectory per sentence (Section 4.5.2, Eq. (38))	New, proposed see the validation and experimentation section described later in this paper for status
Heads and plateaus as formal peak/plateau detection (Section 4.5.3, Eqs. (39)–(41))	New, proposed refer to the validation and experimentation section later in this paper for status
Shared prefix exactness (Section 4.5.4, Eq. (42))	Provable when the full state is identical (graph, timing, arousal history). And the weaker reading of "identical words alone" is not exact as of Revision 3
Gap as shape between two trajectories (Section 4.5.4, Eq. (43))	New, proposed status covered in the validation and experimentation section later in this paper
Reconvergence and the shared word's $\varphi(u)$ value (Section 4.5.5)	New, proposed, and it is derived in two separate ways for status, see the validation and experimentation section described later in this paper (refer Hypothesis 8)
Median line information content (Section 4.5.6, Eq. (44))	An open question see the validation and experimentation section described later in this paper for status
Reverse/retrodictive diffusion (Section 4.5.7)	An open question refer to the validation and experimentation section later in this paper for status
Relationship to the entropy/confidence signal (Section 4.5.8)	Explained see the validation and experimentation section described later in this paper for resolution status. The two signals are shown as able to differ from each other. And they are not shown to correlate or combine in a useful way

5. The Valence-Blindness Problem

Status note (Revision 2). This section’s conclusion is unchanged, but its proof is restated against the new Eqs. (1), (16), (17) rather than the retired Eqs. (1)–(3), and — as the discussion below makes explicit — the replacement mechanism turns out to make the same underlying point even more cleanly than the one it replaces, not less.

The architecture in Sections 3–4 has a structural limitation that is easy to miss on paper and obvious the moment one tries to trace a realistic sequence of user actions through it: nothing in Equation (1) or Equations (16)–(17) depends on whether a selection event was part of an utterance the user meant positively or negatively. The update rule only sees that v was selected following u , and whether that selection matched what diffusion predicted — not what kind of sentence that selection completed.

Definition 1 (Valence blindness). A graph-update rule Φ over selection events (u, v, t) is valence blind if, for any two selection events $e_1 = (u, v, t_1)$ and $e_2 = (u, v, t_2)$ occurring in utterances of opposite sentiment, Φ produces identical changes to (w, TF) .

Proposition 1. The update rule defined by Equations (1) and (16)–(17) is valence blind.

Proof. Equation (1) depends only on $1[\text{selection event at } t + \Delta t]$ and Δt . Equation (16) depends only on $\text{outcome}(u, v, t)$, which is itself defined purely by whether (u, v) carried diffusion mass toward the candidate eventually selected — a statement about predictive success, not about the sentiment of the utterance the selection completed. Equation (17) depends only on whether $TF(u, v, t)$ exceeds θ_{TF} . None of the three take any argument describing the surrounding utterance beyond the identity of the selected token and whether it matched a prediction, so no assignment of sentiment to the surrounding context can change their output. ■

This proof is close to definitional, which is precisely the point: valence blindness is not a subtle emergent failure mode, it is built directly into the update rule as stated, and any evaluation of the architecture that never constructs a valence-conflict test case (Section 10) would not detect it. If anything, Revision 2 sharpens this point rather than weakening it: the retired Eq. (3) at least nominally depended on *usage*, a quantity a sufficiently strained analogy might try to smuggle sentiment into; $\text{outcome}(u, v, t)$ in Eq. (16) depends purely on whether a prediction was confirmed, which has no plausible sentiment-adjacent reading at all. Section 6 remains necessary for exactly the reason it did before.

5.1 A Concrete Instance

Consider a user who selects the candidate word *pizza* twice within a few hours: once completing “I need to buy ___” and once, later, completing “I’ve had enough ___.” Both are selection events on an edge terminating at *pizza*. Intuitively, the first should reinforce an acquisition-oriented association and the second should suppress it — the user is, in context, saying they are done with *pizza*, not that they want more. Under Equation (1), both events apply the identical reinforcement term $+\eta$ to their respective edges; under Equations (16)–(17), both events would identically raise $TF(u, v, t)$ toward transmission-type status, provided each was itself the candidate diffusion had already surfaced. The architecture has no mechanism by which the second event could instead apply a negative or sup-

pressive update. This is not a hypothetical corner case; confirmatory and terminal completions sharing a lexical target are common in ordinary text (requests, complaints, gratitude, cancellations), which is why we treat this as a first-class problem rather than an edge case to be handled later.

5.2 Why This Cannot Be Patched by the Existing Exhaustion Penalty

One might hope the Tabu-style exhaustion penalty already described in Section 3.2 (and Section 5 of the original working notes this paper formalizes) resolves this: after any selection, suppress that node for a cooldown period. It does not resolve the distinction this paper is pointing at, because it suppresses every selection identically, regardless of valence. It correctly stops the keyboard from re-suggesting a word the user just typed a moment ago (useful after either “I need pizza” or “I’ve had enough pizza”), but it applies the same shallow, short cooldown in both cases (Eq. (14)) and does not feed back into whether the underlying association should be strengthened, weakened, or left alone going forward. Section 6 proposes a mechanism that acts on the association itself, not only on short-term re-suggestion.

6. Proposed Extension: Valence-Gated Signed Diffusion

The Valence Gated Signed Diffusion part is a main invented contribution from this paper. This specific part includes several hypotheses created for the IntentSpider system. And also, it includes the relevant theoretical work in mathematics. These points are not presented as already verified points.

6.1 Design Goal

This process has three design goals. First, the process needs to run on every single keystroke inside a phone. And it should not require a repeated connection to a server. Second, it should not require training data other than the data from that single user’s same device. Third is a selection event should be able to push a word connection away from a next word. Which means that, the process should not only stop strengthening that connection. It should push the connection back when the surrounding context depicts that the human is closing the intent, rather than continuing that intent.

6.2 Scoring Valence On Device Through Typing interval patterns

Motivation. In spiders, especially the type “Wandering” Spider, the similar vibration event creates two behavioral responses to opposite sides, which is based on a physical property of the relevant signal. A low frequency vibration between 3 and 4 (used base values for simplification) produces the approach and a fighting or a threat response. And also, a greater vibration from approx. 350 to 450 value produces the escape response (wandering spider research paper. refer [26]). The spider does not need to read a meaning from that signal for this separation. Instead, the spider’s CNS (or central nervous system) uses the physical shape of the signal. This same system is hypothesized in the IntentSpider’s valence signal built from how the human typed a selection rather than the selection process output. Meaning it rejects the rule process. The disadvantage

is that the timing signal can reduce the more clean precision a word list could provide when timing is a unclear or an unrelated signal for the human's intent, which carries a risk of being impractical with the daily life into a some extent. (refer the section no. 12).

Definition. For a selection event at time t , Δt_{avg} represents the average time interval between the keystrokes in the previous context window. Δt_{ref} represents the baseline typing speed of that same user. This is tracked for each human separately, rather than using one fixed constant for all humans. In addition to that, κ is the correction rate value in that similar context window. Which means the simple number of backspaces per each key pressing event. This can be defined as a valence value.

$$\text{val}'(c_t) = \tanh(a \cdot (\Delta t_{ref} - \Delta t_{avg}) - b \cdot \kappa) \quad (20)$$

In this context, $\text{val}'(c_t)$ is a signed value. The sign of this specific value is the main part used by the processes from subsections 6.3 to subsection 6.5. Below in here, the subsection 6.9.1 adds a second axis from the similar typing rate signal. This second axis is unsigned, and it is named as arousal. Which means that, it represents a size and not a direction. Section 6.9.1 builds a further group of processes from this value without changing $\text{val}'(c_t)$.

6.3 Signed Edges

Every edge is extended to carry a signed value $s(u, v, t)$ between -1 and 1 , in addition to the existing raw weight value. For each selection event, this value updates as

$$s(u, v, t + \Delta t) = \text{clip}(s(u, v, t) \cdot e^{-\Delta t / \tau_v} + \eta \cdot \text{val}'(c_t) \cdot \mathbb{I}[\text{selection event}], -1, 1) \quad (7)$$

In here, c_t is the context window around the selection event at time t . And $\text{val}'(c_t)$ is the frequency based signal defined in the equation no. 20. (refer Section 6.2). A confirming selection has a $\text{val}'(c_t)$ value greater than 0. This pushes $s(u, v, \cdot)$ into a positive direction. A closing or rejecting selection has a $\text{val}'(c_t)$ value less than 0. This pushes the signed value into a negative direction, rather than adding the same positive increase from Equation (1). This is the direct correction for the failure mode proved in the main section no. 5. There was a pizza example, the two selection events now create updates with opposite signs. But this result depends on whether typing interval patterns is a useful signal for confirming or rejecting intent in a practical system. Section 6.2 states that limitation, and Section 10 gives the relevant empirical testing process.

6.4 A Third, Suppressive Set, and How It Now Relates to Transmission Fidelity

The two axes are separate by construction. $\text{TF}(u, v, t)$ measures whether an edge reliably predicts the next word the human selects. Which means that, this value is about predictive success and it does not include emotional content (refer Proposition 1). $s(u, v, t)$ measures whether the selection on that edge happened with a positive willness or a negative willness, according to the behavioral timing process previously created. Due to that reason, an edge can have the transmission type and also build a negative valence value at that same

time as well. This depicts a word connection which is predicted reliably and selected in a repeated way, but also selected with a reluctant behavioral signal. The pizza example in that previous section denotes this specific state. After “I have had enough pizza” is typed a sufficient amount of times, the diffusion process can, into a some extent reliably produce the *pizza* again. But the typing interval patterns can depict that the human is closing the intent rather than asking for more.

The suppression rule, stated on the valence axis. An edge which builds a sustained and strongly negative signed weight moves into a third set named as $E_{suppressed}$. In the diffusion process, this set is used as a direct smoothing function process (refer Section 6.5). After that, the relevant word is not removed from the graph. Condition for moving an edge into that set uses only $s(u, v, t)$. It is separate from the TF based role of the words and edges in here.

$$\int_{t-T'}^t \max(0, -s(u, v, \tau)) d\tau > \theta_s \implies (u, v) \text{ moves into } E_{suppressed} \quad (21)$$

Equation (21) is the ordinary route into $E_{suppressed}$. This equation adds the negative valence over a previous time window T' . Which means that, this is a slow building process. Section 6.9.4 below proposes a second and sudden route. In this route, one detected “shock” moves the edge into $E_{suppressed}$ immediately. It does not wait until the value passes θ_s . And this shock route uses a decay constant which is slower than the ordinary $E_{suppressed}$ membership. These two routes are not the same process. One reaches the set through a gradual buildup, and the other reaches that similar set through one sudden event. Section 6.9.4 explains why the motivating analogy includes those two speeds.

A tension from this integration which is not solved here. Sections 3.2 and 3.3 built the TF based typing process with not using ratchet. An edge can move between the transmission type and the capture type as the TF value increases or lowers. The $E_{suppressed}$ process has a different purpose. It is slow and persistent, so a rejected word connection does not return to the ordinary state immediately after the human stops rejecting it. Due to that reason, the architecture includes one process created for fast reversal and another process created for slow persistence. An edge can be typed quickly as transmission or capture, and suppressed slowly at the similar time. Section 6.9.4 adds a third speed into this structure. A detected shock creates an immediate entry into $E_{suppressed}$. This exists alongside the fast TF reversal and the gradual negative value buildup from Equation (21). Which means that, a reader following how the separate system parts change should track three speeds and not only two.

The $E_{suppressed}$ process and its relationship with TF based typing is the most unverified construction in this paper. Non of the CLS based continual learning architectures discussed in Section 2.2, DualNet, CLS ER, and ITDMS, use a third inhibitory memory channel. Each of those architectures uses two memory processes. And non of them includes this specific mismatch between a reliability value that changes freely and a suppression state designed to remain for a longer period. Whether this third set improves calibration, or produces an instability, needs the empirical process in Section 10. Further arguments in this paper cannot verify that point. We include this process because it directly addresses the design gap stated in Section 5. And also, removing the least verified part from a paper which identifies open questions would remove one of the

main purposes of the paper. But Equation (21), $E_{suppressed}$, and its relationship with Equation (17) should be treated as the least verified construction in this document. This is important for any reader who considers this point as a main claim for a patent filing as well.

6.5 Signed Diffusion Read Out

A negative edge should actively cancel a spreading signal. It should not only fail to add more signal. Due to that reason, the unsigned transmission type Laplacian $L = D - W_{transmission}$ from Equation (4) is replaced with a signed Laplacian. This signed Laplacian uses $W_{transmission}$ and $E_{suppressed}$ together. The process follows the signed spectral graph theory from Professor Jerome Kunegis and others (refer [18]).

$$L_{signed} = D_{|\cdot|} - W_{signed}, \quad W_{signed} = W_{transmission} - \gamma \cdot W_{suppressed} \quad (9)$$

In this equation, $D_{|\cdot|}$ is the degree matrix built from the absolute edge weight values. This is required for the signed Laplacian. The γ value is greater than 0, and it controls how strongly a suppressed edge cancels the incoming signal at that node. The heat kernel and personalized PageRank read out processes from Section 4 remain available. Their seed comes from the fan dispersion construction in Section 4.1.1. In here, L_{signed} replaces L , and W_{signed} replaces $W_{transmission}$.

6.6 Algorithm

Algorithm 1. Function typed, valence gated diffusion update and suggestion, per selection event

Require: selection event (u, v, t) , context window c_t , current edge set E (each edge carrying w , TF, s), current $E_{suppressed}$

1. $val' \leftarrow \text{ScoreValence}(c_t) \triangleright$ typing interval patterns + correction rate, Eq. (20), Sec. 6.2
2. update $s(u, v, t)$ via Eq. (7) using val'
3. update $w(u, v, t)$ via Eq. (1) $\triangleright$ raw weight, no channel branch, Sec. 3.2
4. update $TF(u, v, t)$ via Eq. (16) $\triangleright$ transmission fidelity, outcome(u, v, t) from this event
5. **if** $TF(u, v, t)$ greater than θ_{TF} **then** (u, v) is transmission type, included in $W_{transmission}$ (Eq. (17))
6. **else** (u, v) is capture type, left out of propagation (Eq. (17))
7. **end if**
8. **if** suppression condition Eq. (21) is true on $s(u, v, \cdot)$ **then**
9. move (u, v) into $E_{suppressed}$ (Sec. 6.4) $\triangleright$ checked separately from steps 5 through 7, not as an else branch

10. **end if**11. $\triangleright$ On next keystroke, over the last k typed words:12. $s(0) \leftarrow \text{FanDispersionSeed}(u_t, \dots, u_{t-k+1})$ via Eq. (19) $\triangleright$ Sec. 4.1.1, limited to $W_{\text{transmission}}$ 13. $\hat{p} \leftarrow \text{LocalPush}(s(0), W_{\text{signed}}, \alpha, \epsilon)$ $\triangleright$ Sec. 4.3, extended to signed and time changing weights (both open, Secs. 3.3, 6.7)14. **return** top 3 words by $\hat{p}$, and $H(\hat{p})$ as an ambiguity signal (Sec. 4.4)

Step 8 is separate from steps 5 to 7. This is a intentional choice. The reason for this is, the suppression state condition and the transmission or capture type role are measured on two separate axes dimensions or axes planes and due to that reason one update can change an edge from capture type to transmission type in step 5, and also move that edge towards the to execute the process suppression in step 8. This algorithm is not choosing a one result. But instead two results. The W_{signed} construction process in the subsection 6.5 decides how those results work as a combined version in the next step of the diffusion process.

6.7 An Open Question: Does Local Push Still Work Here?

Section 4.3 uses the local push algorithm from Professor Reid Andersen and others (citation no. [1]). Its $O(1/(\alpha\epsilon))$ runtime limiting value is proved for a graph with weights which are not negative values. In that graph, the relevant matrix has a clear random walk process interpretation. The signed values in the equation no. (9) change that interpretation. And Probability mass is not kept in that equal way when a part of that value can be cancelled rather than only moved to another node.

This IntentSpider emotional engine is built on already published mathematics and research by other scientists. and this was the large point and the claim between this emotional engine processes and the runtime viability to run IntentSpider inside a mobile device (examples - iPhone, iPad). It is not the only one now. Section 3.3 adds a second and separate change from the original method by Professor Reid Andersen and others. The limit between the transmission type and capture type changes on every keystroke. Section 4.3 explains that the signed values and the changing edge set add together, rather than replacing each other. A complete mathematical treatment needs to limit the residue related behavior under signed pushes on a graph where the relevant edge set changes at the similar time. As a ojne, this combined problem is complex and difficult than each of them, seperately.

6.8 History Based Task Scheduling Using Weights in Contexts

6.8.1 Motivation

Section 4 ranks every next word using its diffused mass value, p or $s(t)$ in Equations (5) and (4). But a separate choosing question remains. The diffusion process can produce several next words at the similar time. Some of those words can include an already existing graph structure, which is named as residue here. Another word can be new and strongly activated, which is named as fresh. The

system needs a process for deciding which word should be shown first. The valence gate in Sections 6 through 6.5 does not make this decision. It decides whether an edge strengthens or weakens over time. And it does not decide which one of the already active next words receives priority on the current keystroke. This subsection gives a mathematical form to that choosing process. In addition to that, it corrects an earlier informal version which does not produce the intended result when checked directly (refer Remark 1).

6.8.2 The Residue Score

The residue value is defined relative to θ_{TF} . A value of 0 represents no previous reliability buildup. A value moving closer to 1 represents an edge moving closer to the relevant threshold.

For a next word v which is reached through an edge (u, v) that already builds transmission fidelity (refer Section 3.2), we define

$$r(v) = \min\left(\frac{TF(u, v, t)}{\theta_{TF}}, 1\right) \in [0, 1] \quad (22)$$

In here, $r(v) = 0$ represents a next word with no built reliability history. And a $r(v)$ value moving toward 1 represents a word close to the transmission type threshold. The $\min(\cdot, 1)$ process limits the value at 1. Once the edge (u, v) has passed into the transmission type, it competes in the diffusion ranking through its own values (refer Section 4.4). At that point, it is not a residue word competing with a fresh word through this specific rule.

6.8.3 The Support Score

The priority of a fresh next word is built from the previous selection events $e_1, \dots, e_n$ for that word. Each event includes a timestamp t_i and a context window c_i . Two values are defined for every event.

$$R_i = \exp\left(-\frac{t - t_i}{\tau_0}\right) \text{ (pure recency factor),} \quad S_i = \text{sim}(c_i, c_t) \in [0, 1] \text{ (pure context similarity)} \quad (11)$$

These two values are not added directly. Instead, they are combined using a logarithmic opinion pool, or a geometric opinion pool. This is an already existing method for combining two evidence sources through one interpolation value. Professor C. Genest and others introduced this method (refer [7]). And its “external Bayesianity” property makes this specific pooling rule remain consistent under re-parameterization when compared to linear or additive options (refer [8]).

$$w_i(\rho) = R_i^{1-\rho} \cdot S_i^\rho, \quad \rho \in [0, 1], \quad \text{Support}_\rho(v, t) = \sum_i w_i(\rho) \quad (12)$$

The interpolation value is written as ρ and not γ . The γ symbol is already used in Equation (9) for the suppression strength value of the signed Laplacian. Those two values control separate processes. Due to that reason, two separate symbols are used to avoid confusion between them. Section 7.3 below adds another set of gating weights. And for the similar reason, it does not reuse w for those values.

Remark 1 (Correcting an earlier, informal version). An earlier version allowed context similarity to extend the recency decay constant, $\tau_{eff} = \tau_0(1 + \beta \cdot \text{sim})$. Substituting the values directly depicts that this process does not create the intended behavior. When τ_0 increases without a limit, every event's decay value moves toward 1 without depending on its sim value. Which means that, the claimed pure context limit turns into an event count with equal weights. The logarithmic pool in Equation (12) is used because its limiting behavior can be checked directly. Propositions 2 and 3 below give those checks.

6.8.4 Two Limiting Cases

Proposition 2. $\text{Support}_0(v, t) = \sum_i R_i$. This is pure recency, and it does not depend on context.

Proof. Put $\rho = 0$ into Equation (12): $w_i(0) = R_i^1 \cdot S_i^0 = R_i$. ■

Proposition 3. $\text{Support}_1(v, t) = \sum_i S_i$. This is pure context, and it does not depend on recency.

Proof. Put $\rho = 1$ into Equation (12): $w_i(1) = R_i^0 \cdot S_i^1 = S_i$. ■

Those two propositions were also checked with numerical values, rather than leaving them as algebra only. Two created events were used. Event A is recent, with t minus $t_i = 1$, but has a low context relationship, $S_i = 0.10$. Event B is old, with t minus $t_i = 50$, but has a strong context relationship, $S_i = 0.90$. In this test, $\tau_0 = 10$. At $\rho = 0$, the pure recency total is 0.911575. This is exactly equal to $\sum_i R_i$. At $\rho = 1$, the pure context total is 1.000000. This is exactly equal to $\sum_i S_i$. Which means that, the numerical values confirm Propositions 2 and 3 as exact limiting points.

Table 1 depicts another property which is not obvious before checking the values between those limits. The recent but less relevant event does not lose to the old but relevant event at $\rho = 0.5$. The crossover happens between $\rho = 0.5$ and $\rho = 0.75$. In this specific example, recency needs to receive a much lower weight before the context value wins. This behavior comes from the geometric mean process. It is not a separate tuning choice. Due to that reason, $\rho = 0.5$ should not be treated as an equal 50 and 50 balance in the practical process. It can continue to favor recency unless the context relationship is strong. The empirical tuning process in Section 10 should not assume a straight line behavior around this midpoint.

Table 1. The contribution from each event to Support_ρ as ρ moves from pure recency ($\rho = 0$) to pure context ($\rho = 1$), for the two created events above. The crossover between $\rho = 0.5$ and $\rho = 0.75$ is not at the midpoint.

ρ	Event A contribution	Event B contribution	Winner
0.00	0.90484	0.00674	A
0.25	0.52171	0.02291	A
0.50	0.30081	0.07787	A
0.75	0.17344	0.26474	B
1.00	0.10000	0.90000	B

6.8.5 The Arbitration Rule

A fresh next word v_{fresh} receives priority over a residue next word $v_{residue}$ when

$$\text{Support}_\rho(v_{fresh}, t) > r(v_{residue}) \quad (13)$$

Support _{ρ} and $r(v)$, now Equation (22), are defined on approximately comparable value ranges. Those ranges are $[0, n]$ and $[0, 1]$. But a normalization or calibration constant between those ranges is not defined here. It remains as a free parameter, and Section 6.8.6 flags this point rather than assuming a value.

6.8.6 What Is Established Here and What Is Not

The logarithmic opinion pooling process is an established and cited method. Marked or covariate Hawkes processes are also an established broader family of point process models related to this construction. But the use of a logarithmic pool for choosing between recency and context priority in an on device personal intent graph was not found in the literature searched for this paper. Due to that reason, this should be treated in the similar way as the suppressed channel in Section 6.4. It is the least verified single construction in this subsection. It is included because it gives a direct mathematical answer to a relevant design question. And it is not included because the process has already been proved to work.

Three points remain unresolved. The algebra above does not verify those points.

1. The normalization constant between the Support _{ρ} value range and the $r(v)$ value range is not defined. Section 10 gives the empirical process for fitting this value.
2. ρ is a new free parameter. It is added to θ_{TF} , η_{TF} , τ_v , τ_{seed} , η , α , ϵ , and γ which are already listed as unresolved in Section 12. The nonlinear behavior near the midpoint makes ρ more difficult to tune manually than an ordinary straight line mixing weight.
3. Whether this rule improves or reduces the full accuracy is not verified. It has the similar disadvantage stated in Section 6.4. Giving priority to a next word with any existing residue can quietly hold back a truly new word with zero history. And that is the exact state which Section 4 was created to support.

6.9 Details Related to Typing Rate Diff Values, Shared Cluster Reinforcement Process, Growing Radius Value, and Sudden Termination Process

In this section, these six processes came from the motivation by an attack and struggle analogy. Another special note is that they can be considered as invented mechanism as well. This analogy of attacking and struggling of an Orb Spider and Isopod preys inspired these ideas. This is a mathematical hypothesis built from this analogy.

The affective axis in subsection 6.9.1 and the communication channel point in the 6.9.8 are different from those six processes. Professor James Russell's research gives "valence" and "arousal" (These are described as valence and arousal axis and copied here as is.) as two dimensions of affect. As said previously, the term name "arousal" is used in the InterSpider system because it is the name of Professor Russell's second axis of his research. IntentSpider research's name for this second axis as the affective axis is named as "hexarousal" to avoid confusion, and etc. And Professor Joseph's research produces the communication channel difference previously discussed in the previous Section 2.8. (refer citation [29]) But Equation (23) only measures the delta of the typing

rate, or the typing rate difference compared to the ordinary typing rate of that specific human. In a psychological perspective it also does not transform that typing rate into a psychological measurement value. IntentSpider Webnet's input is a typing behavior based value and it is a different point when compared to other researches.

6.9.1 The Typing Rate Difference Signal

Section 6.2 gives the valence signal $\text{val}'(c_t)$. This signal gives whether the update process treats a specific selection as a confirmation result or a rejection result. But it does not give the amount that the current typing rate differs from the ordinary typing rate of that human. For an example, a slow reject and a faster reject with several corrections can both give a negative direction. The direction can be similar. But the amount of the typing rate change can be different. The correction rate is already denoted with κ in the (20) equation as well. The typing rate difference does not have a separate value in that equation.

Professor James Russell's circumplex model denotes valence and arousal as two separate dimensions (refer section 2.8 and citation no. [28]). The first dimension gives the pleasant to unpleasant direction. The second one gives the activated state to deactivated state dimension. As scientists conducted research related to this two dimensional method earlier, this foundation is not invented or novel. But a typing rate value does not become a measuring unit of Professor Russell's axis related to the arousal values, because the value increases. IntentSpider names this second system axis as hexarousal axis. And it names the value as noted below according to the typing data, which creates that axis overtime.

Let $x(t)$ represent the typing rate calculated through a short trailing window ending at t . And let $\mu_{user}(t)$ and $\sigma_{user}(t)$ represent the rolling mean and rolling standard deviation of $x(t)$ for that specific human. The system calculates these two baseline values from the earlier typing data of that human. It does not include the current window when calculating them. Which means that, the current difference cannot change its own baseline. In here, define

$$\text{TRD}(t) = \text{clip}\left(\frac{|x(t) - \mu_{user}(t)|}{\max(\sigma_{user}(t), \epsilon_{rate})}, 0, \text{cap}\right) \quad (23)$$

$\text{TRD}(t)$ type of notations refer to the standardized typing rate difference. ϵ_{rate} represents a small positive value. This positive value is important. It has two purposes. First, it is the value that prevents diving by 0, which results "undefined x/y " errors in C++. and second is when the earlier typing rate has no measured difference across its values. Equation (23) uses an absolute modulus difference. Due to that reason, $\text{TRD}(t)$ has no positive or negative directions. A period faster than the baseline level and a period slower than the baseline level can give a similar $\text{TRD}(t)$ value. The sign of $x(t) - \mu_{user}(t)$ keeps the faster or slower typing direction in places where another process is actively requesting it (or using it). And the sign of $\text{val}'(c_t)$ keeps the separate confirmation or rejection direction and it is used by the signed update process.

This separation is required. $\text{TRD}(t)$ does not prove a psychological state of the human. But rather, it is distance the current typing rate is from the ordinary typing rate of that specific human. Corrections, pauses, a device change, a physical interruption, or a deliberate change in typing speed can create that

difference. For further details, in the subsection 6.9.6 denotes this loss of information. It keeps the separate typing direction inside a larger feature vector.

Δt_{ref} in Section 6.2 and the $\mu_{user}(t)$ and $\sigma_{user}(t)$ values in this section follow a similar per user structure. Non of them use a population typing margin value. A population value can read one human incorrectly when the ordinary typing rate of that human differs from the population value. IntentSpider operates inside a single device for a single human without any external servers (This was previously discussed in depth. Refer to section 3.1). Due to that reason, this baseline value is relevant to this design of the system.

Honest status. cap is a free parameter without an assigned value. It is added to the list in Section 12. This paper also does not decide whether $x(t)$ should come from raw time between keystrokes, a smoothed words per minute value, or another method. proof/validation needs to be done
- remove for now, address later

6.9.2 Compounding Cluster Reinforcement Process Under Repeated and Prolonged activity

Motivation. Equation (1) reinforces one edge after each selection event. But several other next word edges can remain in the active state during a similar short period. The motivation in here is the analogy of a spider receiving continuous and multiple vibration signals from a struggling prey across several threads, rather than receiving one separate strike. Here in IntentSpider system that specific analogy and process becomes a shared and clustered reinforcement process across the edges (connections between words) which were active recently.

Definition. Let $P(t)$ be the *active prey set*. This is the set of next word edges which received a selection or a defined partial activation during the short trailing window w_{active} before t . In this system, we use the already selected diffusion activation margin value value for a partial activity start function. A partial activation does not mean or imply that every diffusion value greater than 0. In that case, $P(t)$ can include a large part of the reachable graph.

$$P(t) = \{(u, v) : \text{lastactivation that is greater than or equal to the specific limit of } (24)$$

With the number of edges in $P(t)$, the diffusion budget amount increases. Which means this is a dynamic value.

$$B(t) = \min(\eta_{cluster} \cdot (1 + \delta)^{|P(t)|-1}, B_{max}) \quad (25)$$

In here, $\eta_{cluster} > 0$ represents the base reinforcement value. $\delta \geq 0$ represents the increase added using the number of active edges. And B_{max} is a value that is assigned to the maximum value for the diffusion budget. When $P(t)$ includes one edge, the budget is $\eta_{cluster}$. When the number of edges increases, the budget also increases until it reaches B_{max} . Which means the equation no (25) allocates a greater reinforcement during a period with several qualifying active edges.

The system shares this budget across $P(t)$. Each edge receives one part based on its diffusion mass with values which are not negative values.

$$\Delta w(u, v, t) = \frac{B(t) \cdot \hat{p}(v)}{\sum_{(u', v') \in P(t)} \hat{p}(v')}, \quad \text{for each } (u, v) \in P(t) \quad (26)$$

Equation (26) needs a denominator value greater than 0. If the total diffusion mass across $P(t)$ is 0, then the system does not apply the shared update at that event. This specific update is added to the reinforcement or further strengthening update, which is in the first equation. (refer Equation (1)), And importantly, it does not change that update.

This is a small note about notations in this section.

1. $\eta_{cluster}$ used for base reinforcement value in equation (25) This is a separate symbol than β_1 , β_2 , and β_3 in the section 7,3 for necessity gating functions.

2. this is different than η , which the equation no. (1) using as learning rate. This should not be confused.

3. $\eta_{cluster}$ is similar to the pattern η_{TF} in the subsection 3.2.

4. The δ and B_{max} symbols are separate from other values in the paper as well.

Honest status. This paper has not tested whether a reinforcement value which increases across several active edges improves prediction. It may instead increase noise during uncertain and fast typing. Section 10 gives a separate hypothesis and an ablation for this specific question. $\eta_{cluster}$, δ , B_{max} , and w_{active} are added to the free parameter list in Section 12. proof/validation needs to be done - remove for now, address later

6.9.3 Growing Engagement Radius Under a Sustained Typing Rate Difference

The motivation in here is the analogy of a spider changing its reach settings when the isopod in the default mode. In IntentSpider, this becomes a gradually increasing scenario in the diffusion distance while a more greater typing rate difference is already active. This is the rising magnitude part of the affective axis or hexarousal. It is a value which moves to more upper levels when the human's typing is in a repeated and a prolonged activity. This event listens and executes TRD(t). Which means that this process identifies the typing behavior of the human into a some extent, but, It does not necessarily imply IntentSpider identified a neurological or a psychological information of the human.

Definition. Let t_i represent the time of the current keystroke. Let θ_D represent the typing rate difference margin value. And let $\ell(t_i)$ represent the number of consecutive keystrokes ending at t_i where TRD remains greater than θ_D .

$$\ell(t_i) = \max\{n : \text{TRD}(t_{i-j}) > \theta_D \text{ for every } j = 0, \dots, n-1\} \quad (27)$$

This definition uses the actual keystroke times t_i . It does not subtract an integer from a continuous time value. After that, the following function gives the gradual increase from this sustained period length.

$$\zeta(\ell) = 1 - e^{-\ell/L_0}, \quad \zeta(\ell) \in [0, 1] \quad (28)$$

The function starts from 0. It moves toward 1 when the number of consecutive keystrokes increases. But it does not reach 1 for a finite ℓ value. The effective teleport value used in Eq. (5) then becomes

$$\alpha_{effective}(t_i) = \alpha_{base} \cdot (1 - \zeta(\ell(t_i))) \quad (29)$$

When they click on keyboard, for each stroke, IntentSpider uses $\alpha_{effective}(t_i)$ in equation no. (5) instead of the ordinary α value. α_{base} represents the α value used when the sustained typing rate difference period is not initiated.

There is also a collision with the notation ρ . In the equation no. (12) in subsection 6.8.3, it already uses ρ for the value of interpolation between the recency value and context value, which are variable values in the logarithmic geometric opinion pool. They should not be confused. Due to that reason, this paper uses $\zeta(\ell)$ for the Equation no. (28). And it uses θ_D instead of θ_A . This is because the executor is the typing rate difference value and not a direct derivation from Professor Russell's arousal axis theories. They should not be confused as well.

The α value controls how frequently the diffusion returns to the starting point (refer Section 4.2). A lower α value lets the signal travel further before that return. When $\ell(t_i)$ increases, $\zeta(\ell)$ increases and $\alpha_{effective}(t_i)$ reduces. Which means that, the diffusion distance increases gradually while the greater typing rate difference is continuing. Hence the direction of the equation matches the growing radius value.

In the equation (29), this also changes $\alpha_{effective}(t_i)$ more closer to 0 when the sustained period becomes longer. The processing amount in the $O(1/(\alpha\epsilon))$ bound increases when the α value reduces. This is the specific disadvantage created by this process. And this disadvantage appears during the same period where the system needs to answer in each of those keystrokes. We also add the human typing behavior as an input to the diffusion process. The shared prefix proposition in Section 4.5.4 considers the words and the graph as the inputs. In the equation (29), it adds the history of the keystroke timing. As in the section 4.5.4, due to that reason, two prefixes with the same words are not allocated to output a similar diffusion output when a human typed those words through different different timing patterns.

Section 4.3 keeps two questions about the $O(1/(\alpha\epsilon))$ bound. The first one comes from signed edge values. The second one comes because the edges in the transmission type edge set are not the same on each keystroke. Those two points ask whether local push finalizes and ends in a fixed amount of time under those changed graph conditions. And the process in this subsection adds a third and separate type of disadvantage. It does not change the α value into an unusable value during one finite keystroke period. Instead, it deliberately reduces the α value used by the bound. And the processing amount increases when that value reduces. Due to that reason, this is a processing disadvantage. And this disadvantage is denoted in the section 9.2 as well.

Honest status. θ_A and L_0 are added to the free parameter list in Section 12. The paper does not decide whether $\alpha_{effective}$ needs a minimum value. Such a value would stop it from moving close to 0. But it would also limit distance a sustained hexarousal period can extend the diffusion. proof/validation needs to be done - remove for now, address later

6.9.4 The Sudden Negative Selection Responses

The motivation in here is the analogy of a physically struggling isopod prey, which makes the spider travel from one specific point to the other and remain within that point for a short period of time. The first response is related to the specific edge connected with that event. The second response changes a specific value in the system for a limited amount of time. Here in IntentSpider system, this specific analogy and process transforms into a pattern taken from $val'(c_t)$, which gives whether the update process reads a specific selection as a confirmation result or a rejection result, and the correction rate, which is already denoted with κ in the (20) equation. But the detected event does not prove that the human experienced a psychological shock. It only gives that pattern from those two values, and it does not necessarily imply IntentSpider identified a neurological or a psychological information of the human.

Let $\mu_{val,user}(t)$ and $\sigma_{val,user}(t)$ represent the rolling mean and standard deviation of the earlier $val'(c_t)$ values for that specific human. The system does not include the current value in this baseline. Let $\epsilon_{val} > 0$ represent a small positive value. This value prevents the standard deviation from becoming 0 in the margin value calculation. The system detects the event when $val'(c_t)$ is lower than the earlier mean of that human by more than k standard deviations

$$shock(t) = \begin{cases} 1 & \text{if } val'(c_t) < \mu_{val,user}(t) - k \cdot \max(\sigma_{val,user}(t), \epsilon_{val}) \\ & \text{and } \kappa(t) > \kappa_{burst_marginvalue} \\ 0 & \text{otherwise} \end{cases} \quad (30)$$

Equation (30) detects an unusual negative value in the statistical values of that human, together with a correction burst. It does not identify the reason which created that pattern. An ordinary typing error, a physical interruption, a device change, or typing with frustration can activate it. Hence, the equation gives an sudden negative selection response inside the system.

Local response. When $shock(t)$ equals 1 during a selection event on edge (u, v) , the system moves that specific edge into $E_{suppressed}$ immediately. It does not wait until the negative integral in Eq. (21) passes θ_s .

$$shock(t) = 1 \text{ on } (u, v) \implies (u, v) \in E_{suppressed}, \text{ with decay constant } \tau_{shock} \quad (31)$$

In here, τ_{shock} represents the persistence period for this route into $E_{suppressed}$. A greater decay constant gives a slower reduction when suppression uses the exponential decay form. Section 6.4 does not give a separate symbol to the ordinary $E_{suppressed}$ decay constant. Due to that

reason, this subsection can state that τ_{shock} is intended to be greater than that ordinary constant. But it cannot give a numerical relationship before the ordinary constant receives a value. Equation (31) creates a longer suppression persistence. It does not create a deeper suppression amount unless the implementation adds a separate suppression magnitude.

During a cool down window w_{shock} after the detected event, the system increases the necessity gating margin value in Eq. (15). Section 7.3 owns that margin value. And it gives this temporary change as Eq. (36).

This corrects the earlier equation direction. Equation (15) displays a suggestion when $N(w, t)$ is greater than κ_N . Reducing κ_N makes that requirement easier to pass. Which means that, the system can display more suggestions. A temporary response which is more careful needs the opposite direction. Due to that reason, the system increases κ_N . This makes the requirement more difficult to pass.

This is a small note about notations in this section.

1. This margin value is κ_N . It is not τ_N .
2. The τ symbols in this paper represent decay constants. This includes τ_v , τ_x , τ_0 , and τ_{seed} .
3. The necessity margin value is not a decay constant. This subsection describes the relationship between the necessity gating algorithm and the overriding process. The local response and the system wide response give a specific form to an earlier design point. That design point was a second signal which can return or cancel a diffusion output with a greater confidence from being displayed. The necessity gate in Section 7 already gives the continuing event executor by using the process denoted there. In addition to that, equation no. (30) adds a separate event event executor. The system does not add the continuing gate and the sudden executor into a one score. The event changes the already existing margin value for a limited amount of time. Then the equation no. (31) is executed to change that edges' state

Honest status. k , $\kappa_{burst_marginvalue}$, w_{shock} , τ_{shock} , and $\Delta\kappa_{shock}$ from Section 7.3 are added to the free parameter list in Section 12. This paper has not tested whether Eq. (30) separates the intended shock events from ordinary frustrated typing. An ordinary period of correction activity may also activate it without carrying the special meaning intended here. proof/validation needs to be done - remove for now, address later

6.9.5 The Timeout Process of the Grace Period and the Settling Event (the function of returning to normal state).

This specific mechanism is inspired by a behavior of a animal predator (Orb Spider).

The definition of this process is, if q represents a one period of activity, then if t_q^{end} represent the time of its last and finalactivation that is greater than or equal to the specific limit A settling event starts when non of the edges in $P(t)$ receives anotheractivation that is greater than or equal to the specific limit during T_{grace} . The default value proposed in here is 60 seconds. This is a design value. It is not a measured result.

Let $\Delta w_q(u, v)$ represent only the weight change added by Eq. (26) during period q . And let $\Delta \alpha_q(t') = \alpha_{base} - \alpha_{effective}(t')$ represent only the teleport reduction added by Eq. (29) during that period. The system keeps these values separately. This is because it cannot reduce the changes from one specific period later if it has already combined them permanently with the ordinary updates.

A confirmed period is one where $\text{val}'(c_t)$ is greater than 0 at the final selection event. Otherwise, the period is unconfirmed. Define $m_q = 1$ for a confirmed period. And define $m_q = m_{unconf} > 1$ for an unconfirmed period. For $t \geq t_q^{end} + T_{grace}$, the return process is

$$\begin{aligned}\Delta w_q(u, v, t) &= \Delta w_q(u, v, t_q^{end}) \cdot e^{-(t-t_q^{end}-T_{grace})m_q/\tau_v}, \\ \Delta \alpha_q(t) &= \Delta \alpha_q(t_q^{end}) \cdot e^{-(t-t_q^{end}-T_{grace})m_q/\tau_\alpha}.\end{aligned}\tag{32}$$

The first line returns the reinforcement added during that specific period toward 0. The second line returns $\alpha_{effective}$ toward α_{base} . It does this by reducing the teleport difference added during that period. τ_v and τ_α keep the weight return and the radius return as two separate speeds. An unconfirmed period uses $m_{unconf} > 1$. Which means that, its temporary changes reduce faster than the temporary changes from a confirmed period.

This process creates a bookkeeping requirement. The implementation needs to keep the update source, the related activity period, the final event, and the two temporary values separately. Without that information, Eq. (32) cannot find which part came from Eq. (26), which part came from Eq. (29), and which part belongs to the ordinary system update.

Honest status. This is not only an equation. It also creates a bookkeeping requirement. The implementation has to find which exact weight changes and which exact radius changes came from one specific period after that period has already ended. Then it has to decay those changes at a separate rate from the ordinary background decay. This gives a non-trivial engineering disadvantage. This paper keeps that point instead of removing it. T_{grace} and m_{unconf} are added to the free parameter list in Section 12. proof/validation needs to be done - remove for now, address later

6.9.6 Emergent Per User Typing Sub State Clustering

Motivation. TRD(t) is a one scalar magnitude. It does not keep whether the typing becomes faster or slower. And a similar magnitude can appear with a different correction rate, pause structure, or $\text{val}'(c_t)$ direction. For an example, fast typing with few corrections, fast typing with several corrections, and a long pause followed by a selection are separate typing states. Equation (23) can place them under a similar magnitude. This subsection keeps those measured patterns as typing sub states. It does not name them as emotional states without a separate test.

Definition. Let

$$f(t) = (z_x(t), \text{TRD}(t), \tilde{\kappa}(t), g_{\text{pause}}(t), \text{val}'(c_t)), \quad z_x(t) = \frac{x(t) - \mu_{\text{user}}(t)}{\max(\sigma_{\text{user}}(t), \epsilon_{\text{rate}})} \quad (33)$$

In here, $z_x(t)$ keeps the faster or slower typing direction. And $\text{TRD}(t)$ keeps the limited absolute magnitude from Eq. (23). $\tilde{\kappa}(t)$ represents the correction rate after a per user normalization. $g_{\text{pause}}(t)$ represents the pause pattern feature. And $\text{val}'(c_t)$ is the selection direction already used in Section 6.2. These separate parts need comparable scaling before one distance margin value combines them. Otherwise, one part with a greater numeric unit can decide the cluster distance by itself.

An online clustering process can run per user and fully inside the device. It groups the recent $f(t)$ values using a distance margin value δ_{cluster} . The number of groups is not fixed before the typing data exists. But the meanings of those groups remain limited to patterns in the measured typing features. Equation (33) alone cannot name one group as rushed, happy, frustrated, or another emotional state.

This point is not specified. This paper does not define $g_{\text{pause}}(t)$. And it does not select the normalization for $\tilde{\kappa}(t)$, the distance function, the clustering algorithm, or the period used by that process. Due to that reason, Eq. (33) gives the inputs which need to remain separate. It does not complete the clustering process. The distance margin value δ_{cluster} cannot receive a value with a clear meaning before those missing definitions are given.

Honest status. δ_{cluster} does not have an assigned value. The online clustering algorithm is also not selected. In addition to that, this paper does not know whether the sub states found by that process will match emotional states a human can recognize. They may instead come from the clustering method itself. This stays as an open empirical question. proof/validation needs to be done - remove for now, address later

6.9.7 A Nonlinear, Multi Dimensional Behavioral Weight

Motivation. Sections 6.9.2 and 6.9.3 use a one scalar typing rate difference. Equation (33) keeps more than one measured behavior. This subsection gives a one bounded value from those features for a place where the system needs a multiplier with values which are not negative values. This value is a behavioral interaction weight. It is not a direct measurement of a psychological state.

Definition. Let $\tilde{g}_{\text{pause}}(t)$ represent a pause feature normalized into a value from 0 to 1. Let $\tilde{\kappa}(t)$ also remain from 0 to 1. And let $\widetilde{\text{TRD}}(t) = \text{TRD}(t)/\text{cap}$ when cap is greater than 0. In here, define

$$\xi(t) = \text{clip}\left(\tanh(\nu_1 \widetilde{\text{TRD}}(t) + \nu_2 \tilde{\kappa}(t) + \nu_3 \tilde{g}_{\text{pause}}(t) + \nu_4 |\text{val}'(c_t)|), 0, 1\right) \quad (34)$$

The ν_1 to ν_4 coefficients have values which are not negative values. Equation (34) therefore gives 0 when all four input magnitudes are 0. And it moves toward 1 when their weighted combination increases. The absolute value of $\text{val}'(c_t)$ keeps the confirmation or rejection direction

outside this magnitude. This is required if $\xi(t)$ replaces another multiplier which cannot use a negative value. If the system uses $\text{val}'(c_t)$ with its sign inside the sum, a negative selection can reduce or reverse the weight. That would create a separate process.

Notation correction. An earlier name for this value used $\alpha_{\text{weight}}(t)$. But the α name group already includes α , α_{base} , and $\alpha_{\text{effective}}$. Those values belong to the teleport process. The behavioral weight is separate from that process. Due to that reason, it is named $\xi(t)$.

The earlier coefficient names w_1 to w_4 also connect with notation already used in this paper. This paper uses $w(u,v,t)$ for edge weights. And Eq. (12) uses $w_i(\rho)$ for the logarithmic pool. Section 7.3 uses β_1 , β_2 , and β_3 for necessity gating. Hence, those names are not reused in here. The coefficients in Eq. (34) are named v_1 to v_4 .

An open integration question. Equation (27) uses $\text{TRD}(t)$ to find a sustained period. And Eq. (25) uses the number of active edges instead of $\xi(t)$. This paper does not decide whether $\xi(t)$ should replace $\text{TRD}(t)$, multiply the budget, or remain unused by those two processes. It also does not define when enough per user data exists to start using the pause feature and its normalization. A change from $\text{TRD}(t)$ to $\xi(t)$ during one specific keystroke can create a sudden system change. Non of those integration choices are selected by Eq. (34).

Honest status. v_1 to v_4 are added to the free parameter list in Section 12. The integration question above also remains open. proof/validation needs to be done - remove for now, address later

6.9.8 The Communication Channel Calibration

Established and cited. Unlike Sections 6.9.2 to 6.9.7, the communication channel difference does not come from the spider analogy. Professor Joseph Walther's Hyperpersonal Model [29], together with the cues filtered out and social information processing research discussed in Section 2.8, showed that computer mediated communication differs from face to face communication. And this research gives one specific point. The way affective information appears and gets read is depending on the communication channel. Text messaging, email, note taking, and another written system do not give identical typing conditions.

The design implication in here is limited. A typing rate baseline calculated in one channel should not be transferred directly into another channel. This is because the ordinary typing rate and the pause behavior of the same human can change with the task and the communication system. Let $\mu_{\text{user,channel}}(t)$ and $\sigma_{\text{user,channel}}(t)$ represent the earlier baseline values calculated for one human inside one channel. An illustrative calibrated value is

$$\text{TRD}_{\text{channel}}(t) = c_{\text{channel}} \cdot \text{clip}\left(\frac{|x(t) - \mu_{\text{user,channel}}(t)|}{\max(\sigma_{\text{user,channel}}(t), \epsilon_{\text{rate}})}, 0, \text{cap}\right) \quad (35)$$

In here, c_{channel} represents a calibration value estimated for that specific channel. Equation (35) does not create that value. A channel specific baseline already removes one part of the ordinary difference be-

tween channels. Due to that reason, $c_{channel}$ should only represent a remaining difference measured for that channel. The paper should not assign it without data. And the system should not use it to turn TRD into a measurement of Professor Russell's arousal axis.

This equation also does not claim one fixed behavior when the human moves between channels. It gives a warning that the system needs separate data for each channel. If one channel does not have enough earlier typing data, the system cannot claim that its baseline for that channel is calibrated. In that state, the system can use a per user value which is clearly named as uncalibrated. Or it can avoid the channel adjustment until the required data becomes available.

7. Necessity Gating and Deciding Whether to Suggest

7.1 Motivation

The architecture in Sections 3-6 decides which next word should be suggested. But it does not answer a separate question which comes before displaying that word. This question is whether the system should display a suggestion or remain without a suggestion. A system which always displays its first ranked next word can show a word even when that word has weak support. Repeating this process can reduce the usefulness of the keyboard. Due to that reason, this section gives a gating process for that specific decision. This process is added after the ranking processes in Sections 3-6. It does not change how those sections rank the next words.

7.2 The SCAMPER Generation Method

The process below was generated using SCAMPER, or Substitute, Combine, Adapt, Modify, Put to another use, Eliminate, and Reverse. SCAMPER is a structured creativity checklist [23] built after Professor Alex Osborn's earlier Applied Imagination checklist [24]. This method can generate a system design. But the method does not prove that the generated design works. Due to that reason, Sections 7.5 and 10.4 test the process based on its own results. The paper does not accept it because of the method which produced it.

Naming the method in here is a deliberate difference from Sections 3-6. Those sections build from cited and already established processes and adapt them into a different setting. This section starts from an acknowledged creativity method. Which means that, Section 7.6 keeps it under a stricter not yet established level than the processes in Sections 3-6.

- **Substitute.** The implicit rule which always displays the first ranked next word is replaced by a continuing necessity score. The system compares that score with a threshold.
- **Combine.** The score combines signals which the architecture already produces. This includes the entropy confidence signal in Section 4.4 and the valence based urgency from the seed context in Section 6.2.

Which means that, this step does not create another input signal without an earlier source.

- **Adapt.** The exhaustion value in Section 3.2, Eq. (14), becomes an input which reduces the necessity score. A pattern which was satisfied recently is therefore less likely to pass the threshold again during its suppression period.
- **Modify and Magnify.** One possible extension changes the threshold for each human through the earlier data of that human, instead of using one population constant. This is the least decided part of this proposal. This section does not give a process for making that change.
- **Put to another use.** One considered use was inverting the same score and using it to move an edge faster than the ordinary process in Eq. (3)/(8). This version does not use that extension. It adds another process without a required role in the current system.
- **Eliminate.** Another option removes necessity gating and lets the human ignore an incorrect suggestion. This remains as the null comparison B0 in Section 10.2. It is not removed from the evaluation.
- **Reverse.** The reversed question is whether the system remaining without a suggestion gives information to the human. This stays as an open question. Section 10 measures the quality of displayed suggestions. It does not measure the human facing result of an absent suggestion (refer Section 13).

7.3 The Proposed Process

Let w represent one next word at time t . The necessity score combines three quantities which the architecture already produces.

$$N(w, t) = \beta_1 \cdot (1 - H_{norm}(p)) + \beta_2 \cdot \max(0, \text{valence}_{urgency}(c_t)) - \beta_3 \cdot \text{exhaustion}(w, t) \quad (15)$$

Two parts of Eq. (15) need a separate definition because the earlier signals do not give those parts directly.

- **The normalized entropy value.** Let S_k be the set of k next words used by the gating process, where $k > 1$. The diffusion values across S_k are normalized so that their total is 1. In Eq. (15), p refers to that normalized distribution. Define $H_{norm}(p) = H(p)/\log(k)$. This is the entropy value from Section 4.4 divided by the entropy of a uniform distribution across those k next words. Hence, $H_{norm}(p) \in [0, 1]$. And $1 - H_{norm}(p)$ gives a greater value when the diffusion result gives more support to a limited number of next words. If the total diffusion value across S_k is 0, the system does not apply Eq. (15) and does not display a suggestion. The entropy value $H(p)$ in Section 4.4 did not include this normalization. This section adds it because Eq. (15) combines entropy with two inputs scaled from 0 to 1.
- **The valence urgency value.** Define $\text{valence}_{urgency}(c_t) = \max(0, -\text{val}'(c_t))$, where $\text{val}'(c_t) \in [-1, 1]$ is the typing frequency based context valence from Section 6.2, Eq. (20). This gives a greater urgency value when the valence direction is more negative. And a valence value which is 0 or greater adds no urgency. This use remains separate from the

sign direction used by Eq. (7) for the edge update. An earlier description only named this as a simple transform with values which are not negative values. This definition gives the exact transform used by Eq. (15).

The $\text{exhaustion}(w, t)$ value comes from Eq. (14) in Section 3.2. More than one edge can end at the same next word w . Due to that reason, this section uses the greatest exhaustion value across the edges which end at w .

$$\text{exhaustion}(w, t) = \max_{(u, w) \in E} \text{exhaustion}((u, w), t)$$

This gives one value to Eq. (15). And it prevents the gate from ignoring an active exhaustion value only because another edge ends at the same word. This aggregation choice is a design choice in this section. Section 10.4 needs to test its result together with the other parts of the gate.

β_1 , β_2 , and β_3 represent the three weights. κ_N represents the threshold. Similar to the other free parameters in this paper, their values need to come from the experiments in Section 10.5. The other free parameters include θ_{TF} , η_{TF} , τ_v , τ_{seed} , η , α , ϵ , γ , ρ , τ_x , a , b , and Δt_{ref} . T and the earlier promotion threshold θ were removed in Section 3.3. This section does not assign values to β_1 , β_2 , β_3 , or κ_N without that data.

The notation uses κ_N instead of another τ or θ symbol. The τ name is already used by the decay constants τ_v , τ_0 , and τ_x . And the θ name was used by the promotion threshold in Eq. (3)/(8). The weights use β_1 , β_2 , and β_3 instead of w_1 , w_2 , and w_3 . This paper already uses w for the edge weights w_{stable} and $w_{volatile}$. Equation (12) also uses $w_i(\rho)$ for the support pool weights. Reusing w for a third type of value can connect three separate processes under a similar notation. Due to that reason, this section uses the β symbols.

The system displays a suggestion only when $N(w, t) > \kappa_N$. When the score is equal to or lower than the threshold, the system displays no suggestion.

Extension from Revision 3. Section 6.9.4 changes this threshold for a limited cool down window w_{shock} after a detected shock retreat event in Eq. (30).

$$\kappa_N(t) = \kappa_N + \Delta\kappa_{shock} \cdot \mathbb{I}[t \text{ is within } w_{shock} \text{ of the most recent shock event}] \quad (36)$$

In here, $\Delta\kappa_{shock} > 0$ is another free parameter. The system uses $\kappa_N(t)$ instead of the ordinary κ_N value on every keystroke. Outside a cool down window, $\kappa_N(t) = \kappa_N$. Which means that, the ordinary gate remains unchanged outside that limited period.

Equation (36) also corrects the earlier direction. Equation (15) displays a suggestion when $N(w, t)$ is greater than the threshold. Lowering the threshold makes the requirement easier to pass. It can therefore display more suggestions. A response which is more careful after the event needs a greater threshold. Due to that reason, Eq. (36) adds $\Delta\kappa_{shock}$ instead of subtracting it.

7.4 Relationship With Residue Priority Arbitration and the Full Pipeline

The arbitration process in Section 6.8 and the gate in this section answer two separate questions. And the system applies them in a sequence. Arbitration in Eq. (13) decides which one ranks first across several active next words. Necessity gating decides whether that first ranked next word passes the requirement for being displayed. Which means that, one next word can win the arbitration process and then fail the necessity gate.

Algorithm 1 in Section 6.6 is changed based on this ordering. Local push first produces the ranked next words. This was the final step in the earlier algorithm. After that, the system applies Eq. (15) to the first ranked next word before returning a result. The changed end of the algorithm is given below.

Algorithm 1 update. Function typed and valence gated diffusion update and suggestion for each selection event

Steps 1-13 remain as given in Section 6.6. The system calculates valence from the typing pattern through Eq. (20). It updates the signed edge weight through Eq. (7) and the raw weight through Eq. (1). It updates transmission fidelity and checks the role assignment through Eq. (17), or checks the separate suppression through Eq. (21). After that, it builds the fan dispersion seed through Eq. (19) and runs local push.

14. $\text{nextWords} \leftarrow$ top 3 words by $\hat{p}$, ranked through Eq. (13) when the residue next words are competing (refer Section 6.8.5)
15. $w^* \leftarrow$ first ranked word in nextWords
16. calculate $N(w^*, t)$ through Eq. (15)
17. if $N(w^*, t) \leq \kappa_N$ then return $\emptyset$ and stop (no suggestion is displayed, refer Section 7.3)
18. return nextWords and $H(\hat{p})$ as the ambiguity signal (refer Section 4.4)

Steps 14-18 replace the earlier step 14 in Section 6.6. The earlier step returned the top 3 words by $\hat{p}$ and the $H(\hat{p})$ value. This update separates that one step into ranking, gating, and return steps. It does not change how the system calculates $\hat{p}$. And it does not change how Eq. (13) resolves the competition between the active next words.

This ordering has one specific reason. The system applies the gate only to the first ranked next word after arbitration. In Section 6.8.5, Eq. (13) can move a newly active next word before another word which has a greater residue. That newly moved word has less earlier information supporting it. Due to that reason, it is a specific case where the necessity gate needs to check the result after arbitration.

Another considered ordering applied $N(\cdot, t)$ to every active next word before arbitration. After that, it would apply arbitration only to the words which passed the gate. This version does not use that ordering. In that case, the necessity score also becomes a second arbitration signal.

And that would combine the result of Hypothesis 5 about gating with the result of Hypothesis 4 about arbitration in Section 10.4.

Status note from Revision 3. The steps above show the Revision 2 integration. The complete pipeline below replaces that update because it also includes the processes from Section 6.9.

Algorithm 1 complete update. Function typed, valence and hexarousal gated diffusion update and suggestion for each selection event

1. $\text{val}' \leftarrow \text{ScoreValence}(c_t)$ through Eq. (20) (unchanged, refer Section 6.2)

1a. $\text{TRD}(t) \leftarrow$ the standardized typing rate difference in Eq. (23). This gives the typing input used by the affective axis, or hexarousal (new, refer Section 6.9.1).

2. update $s(u, v, t)$ through Eq. (7) (unchanged, refer Section 6.3)

3. update $w(u, v, t)$ through Eq. (1) (unchanged, refer Section 3.2)

3a. update the active prey set $P(t)$ through Eq. (24). If $|P(t)| > 1$, calculate the reinforcement budget $B(t)$ through Eq. (25) and share it across $P(t)$ through Eq. (26) (new, refer Section 6.9.2).

4-7. update $\text{TF}(u, v, t)$ and the role assignment through Eqs. (16)-(17) (unchanged, refer Section 3.3).

8-10. check the ordinary suppression through Eq. (21) (unchanged, refer Section 6.4).

8a. calculate $\text{shock}(t)$ through Eq. (30). If $\text{shock}(t)=1$, move the specific edge (u, v) connected with the event into $E_{\text{suppressed}}$ immediately through Eq. (31). Record the shock time for step 17 (new, refer Section 6.9.4).

11. next keystroke

11a. update the streak length $\ell(t)$ through Eq. (27) using θ_D . Calculate $\zeta(\ell(t))$ through Eq. (28). After that, calculate $\alpha_{\text{effective}}(t)$ through Eq. (29) (new, refer Section 6.9.3).

12. $s(0) \leftarrow \text{FanDispersionSeed}(\dots)$ through Eq. (19) (unchanged, refer Section 4.1.1)

13. $\hat{p} \leftarrow \text{LocalPush}(s(0), W_{\text{signed}}, \alpha_{\text{effective}}(t), \epsilon)$. The $\alpha_{\text{effective}}(t)$ value replaces the ordinary α value in this step (refer Section 6.9.3)

14. $\text{nextWords} \leftarrow$ top 3 words by $\hat{p}$, ranked through Eq. (13) (refer Section 6.8.5)

15. $w^* \leftarrow$ first ranked word in nextWords

16. calculate $N(w^*, t)$ through Eq. (15)

17. calculate $\kappa_N(t)$ through Eq. (36). Use the ordinary κ_N value unless a shock time recorded at step 8a remains inside w_{shock} . During that period, use $\kappa_N + \Delta\kappa_{shock}$ (new, refer Section 7.3)
18. if $N(w^*, t) \leq \kappa_N(t)$ then return $\emptyset$ and stop (no suggestion is displayed)
19. return nextWords and $H(\hat{p})$ as the ambiguity signal (refer Section 4.4)

This is the second change to the end of Algorithm 1. Revision 2 replaced the one return step in Section 6.6 with steps 14-18. This revision adds steps 1a, 3a, 8a, and 11a. These added steps calculate TRD(t), apply the compounding reinforcement, detect the shock event, and change the diffusion radius. The lettered steps keep the earlier step numbers without moving every unaffected step. And the gating process now uses the temporary threshold in Eq. (36) instead of one constant threshold.

Non of these changes alter how Eq. (13) resolves a competition between active next words. The only change inside local push is the use of $\alpha_{effective}(t)$ in place of the ordinary α value. And the earlier reason for applying gating after arbitration remains unchanged.

The settling event in Section 6.9.5 does not appear as a keystroke step in the algorithm above. It starts after T_{grace} passes without another qualifying activation across the edges in $P(t)$. Which means that, it is not an event created by one keystroke. The implementation can run it as a separate check during an inactive period. The exhaustion decay in Eq. (14) follows a similar background structure. Due to that reason, the settling check does not need to become a branch inside the per keystroke process above.

7.5 What Is Established Here and What Is Not

Part of this proposal	Status
The three inputs, entropy, valence, and exhaustion	Each input is already established and cited, or defined earlier in this paper (refer Sections 4.4 and 6.2, and Section 3.2 Eq. (14)).
Combining more than one confidence signal into one gating score before the system acts	The general process already exists in decision theory and recommendation systems through confidence gating and abstention. The specific combination in Eq. (15) is not established by that general process.
The specific weighted equation in Eq. (15), including the H_{norm} and $valence_{urgency}$ definitions in this section	New process proposed by this section. It needs the tests in Sections 7.5 and 10.4.
A per user changing threshold from the Modify step in Section 7.2	The least decided part. This section does not give a process for setting or changing it.
Placing the gate after arbitration in Algorithm 1	A system design choice made in this section. The reason is given in Section 7.4. It does not have a separate test outside the complete gating ablation.

7.6 Why This Stays Hedged

Equation (15) can be written with a confidence which the current data does not give. For an example, the paper could remove the word proposed. It could assign values to κ_N , β_1 , β_2 , or β_3 before the experiments. Or it could include this section inside the novelty claim in Section 11 as an already established result. Non of those changes would add evidence to the equation.

The three inputs have earlier sources. And confidence gating and abstention are already existing system patterns. But those points do not prove that this specific weighted combination improves IntentSpider. They also do not decide the threshold or the three weights. Due to that reason, this section keeps the process as proposed until the experiments in Section 10 give the result. This separation also protects the claims which have a different evidence level, including the mathematical proofs in Sections 4 and 5.

8. Privacy Considerations

The architecture’s privacy property is, by design, the simplest kind available: data locality. Because the graph, the per-user typing-cadence baseline (Δt_{ref} , Section 6.2), and every update rule in Sections 3-7 run entirely on-device with no server component, no raw keystroke, no aggregated gradient, and no derived graph statistic needs to leave the device for the system to function. This is a strictly weaker (and easier to argue for) property than a formal differential-privacy guarantee [6]: it says nothing about what could be inferred if the device itself, or a backup of it, were compromised, and it offers no protection if a future version of the system were changed to sync graph state to a cloud service for cross-device continuity.

We deliberately do not claim a formal privacy proof for two reasons. First, none has been constructed: extending differential-privacy guarantees to an evolving, signed, function-typed graph is a nontrivial research question in its own right, not a checkbox. Second, the two hardest versions of that question - adding calibrated noise to a representation with intrinsic non-Euclidean structure without formally proving what that noise guarantees, and bounding what a general trait (rather than a single record) can be inferred from accumulated structure over time - are each substantial enough to be their own paper, and we do not want to attach an informal treatment of either to this one merely for completeness. (The second of these traces back to an early brainstorming direction for this project - inferring a user’s gender or other protected attributes from accumulated graph structure - that was deliberately redirected toward this open question rather than built as a feature; Appendix A records that provenance directly.) We list both explicitly as future work in Section 12 and decline to cite specific mechanisms for them here that we have not verified apply to this setting.

New consideration (Revision 2). Replacing a lexicon-based valence scorer with a typing-cadence-based one (Section 6.2) removes a depen-

dency on word content, which sounds like a straightforward privacy improvement, and in one specific sense is: nothing about *what* was typed needs to be interpreted to produce $\text{val}'(c_t)$. But it introduces a different kind of sensitive signal in its place. Inter-keystroke timing and correction rate are exactly the features keystroke-dynamics research uses for *individual re-identification* [27] — the same Δt_{avg} , Δt_{ref} , and κ statistics this architecture stores for valence-scoring purposes are, in that adjacent literature, a behavioral fingerprint good enough to authenticate a specific person. This paper’s design keeps those statistics on-device and never transmits them, for the same data-locality reason it never transmits the graph itself, so this is not a new transmission risk under the threat model already stated above. It is, however, a new *composition* risk worth naming rather than leaving implicit: a device-level compromise of this system’s stored state would now expose not only what a person associates with what (the graph), but a behavioral biometric trace of how they type, which is a categorically different kind of sensitive data than either a word graph or a word-sentiment lexicon ever stored. Any real implementation should treat Δt_{ref} and related per-user timing statistics with at least the same at-rest protection recommended below for the graph itself, not as an incidental, lower-sensitivity by-product of valence scoring.

What we can say concretely is a design recommendation: the graph state described in Sections 3, 6, and 7 is a compressed, structured record of a person’s private associations, including, plausibly, health conditions, relationships, and situational vulnerabilities inferable from what a person types about and when. Any real implementation should treat that graph itself as sensitive at rest (encrypted, without cloud backup by default) even though it is never transmitted during normal operation, and should give the user an explicit, low-friction way to inspect and delete parts of it.

New consideration (Revision 3). Section 6.9.1’s arousal baseline, μ_{user} and σ_{user} of typing rate, is additional per-user behavioral-biometric data of the same category as Δt_{ref} above, for the same reason: it is a self-relative statistic derived purely from timing, stored on-device, and never transmitted, but a device-level compromise would expose it alongside everything else. It should receive the same at-rest protection already recommended for Δt_{ref} and the graph itself, not weaker protection on the assumption that it is somehow more incidental. Section 6.9.6’s emergent sub-state clusters are a distinct and, we think, more sensitive case: where Δt_{ref} and Arousal(t) are single scalars, a discovered set of per-user typing sub-states is, in effect, a rudimentary emotional profile of that specific person — however many distinguishable excited states their own typing supports, characterized by whatever features happen to separate them. This is a qualitatively different kind of derived data from a scalar biometric, closer in kind to the graph’s own “compressed record of private associations” than to a single timing statistic, and any real implementation should extend the inspect-and-delete recommendation above to cover discovered sub-state clusters specifically, not only the graph.

9. On-Device Feasibility: Order-of-Magnitude Estimates

We give back-of-envelope estimates here, explicitly labeled as such. None of the numbers below are measured; they are illustrative calculations under stated assumptions, intended to establish plausibility, not to substitute for the profiling a real implementation would require.

9.1 Memory

Suppose a heavy typist accumulates on the order of $|V| \approx 5 \times 10^4$ distinct tokens over several years, with an average node degree of $d \approx 20$. (*Status note, Revision 2: the earlier estimate scoped d “across both channels combined”; with a single edge set, Section 3.2, this is now simply the per-node degree.*) Each edge stores a signed weight, a raw weight, a transmission-fidelity score, and a last-used timestamp; at 4 bytes each, that is 16 bytes per edge, plus modest overhead for the adjacency structure itself (say, 8 bytes for pointers/indices), for roughly 24 bytes per edge:

$$|V| \times d \times 24 \text{ bytes} \approx 5 \times 10^4 \times 20 \times 24 \approx 24 \text{ MB.}$$

The frequency-derived valence signal of Section 6.2 replaces the earlier lexicon (a few thousand entries, well under 1 MB) with a handful of scalars per user — Δt_{ref} and a short rolling window of recent Δt and backspace counts — negligible even against the lexicon it replaces. Both totals are trivial against typical modern phone storage and RAM budgets, though we have not measured actual overhead from a real data structure implementation, garbage collection, or OS-level constraints.

Extension (Revision 3). Section 6.9 adds a handful more per-user scalars (μ_{user} , σ_{user} for typing rate; $\mu_{val,user}$, $\sigma_{val,user}$ for valence; the current streak length $\ell(t)$; a shock-timestamp) and, per implicated edge, a shock flag and its own decay clock (Section 6.9.4) — all negligible in the same sense as Δt_{ref} above, on the order of tens of bytes total. Section 6.9.6’s discovered sub-state clusters are the one component here worth flagging separately: storing k discovered centroids over a five-dimensional feature vector (Eq. (33)) is itself trivial, but the online clustering process that discovers them is a standing background computation this estimate does not attempt to bound, since no clustering algorithm is fixed in Section 6.9.6.

9.2 Per-Keystroke Latency

The dominant cost per keystroke is the local push computation (Section 4.3), bounded at $O(1/(\alpha\epsilon))$ node touches in the unsigned, fixed-membership case (Sections 6.7 and 3.3 together note this bound is not yet proven once the edge set is both signed and time-varying, which it now is). For typical personalized-PageRank parameters ($\alpha \approx 0.15$, $\epsilon \approx 10^{-5}$), this bound suggests on the order of 10^5 - 10^6 elementary node-touch operations in the worst case, each touching a handful of floats - almost certainly sub-millisecond to low-single-digit-millisecond work on current mobile CPUs, but again, this is an extrapolation from the com-

plexity bound, not a measurement, and worst-case bounds on real personal graphs with heavy-tailed degree distributions can behave differently from the bound’s typical case. Constructing the fan-dispersion seed (Eq. (19), Section 4.1.1) adds $O(k)$ work for k recent tokens plus one dispersion-score lookup per token, negligible against the local push cost itself.

The necessity score of Eq. (15) (Sec. 7.3) adds negligible cost on top of this: it is a fixed weighted sum of three already-computed scalars per candidate, $O(1)$ per candidate against the $O(1/(\alpha\epsilon))$ cost of the local push step itself, so gating does not change the order-of-magnitude estimate above.

Extension (Revision 3). Two of Section 6.9’s mechanisms change this picture, in different ways. First, compounding reinforcement (Eq. (26), Section 6.9.2) and shock detection (Eq. (30), Section 6.9.4) are each $O(1)$ or $O(|P(t)|)$ per keystroke — negligible in the same sense as the necessity score above, since $|P(t)|$ is bounded by however many candidates can plausibly be simultaneously “live” (small in practice). Second, and materially different: the $O(1/(\alpha\epsilon))$ bound itself is no longer evaluated at a fixed α once growing radius (Section 6.9.3) is active, and the estimate above (10^5 – 10^6 node touches at $\alpha \approx 0.15$) is a lower bound, not the typical case, once $\alpha_{\text{effective}}(t)$ begins shrinking under a sustained-arousal streak. As $\alpha_{\text{effective}} \rightarrow 0$, the same bound grows without an upper limit — precisely the cost-inflation concern flagged in Section 6.9.3 and Section 4.3, arriving here as a concrete latency risk rather than only a theoretical one. Whether a practical floor on $\alpha_{\text{effective}}$ is needed to keep worst-case latency within a usable range during exactly the excited typing bursts this mechanism targets is an open implementation question this estimate does not resolve.

9.3 Comparison to Existing Approaches

Table 2 summarizes, at a qualitative level, how the proposed architecture relates to the two deployed approaches discussed in Section 2 and to the adjacent research architectures discussed in Section 2.2. The table does not include a necessity-gating column, since none of the compared systems implement any form of confidence-based suppression; the relevant comparison for that question is B0 versus B4 in Section 10, not this table.

Table 2. Qualitative comparison. The last row’s final column is the honest and most important entry in this table.

Approach	On-device, no server	Handles cold-start	Handles valence	Empirically validated
n-gram + smoothing [14,17]	Yes	No	No	Yes, extensively
Federated RNN keyboard [10]	Partial (gradients only)	Weak	No	Yes
DualNet / CLS-ER / ITDMS [21,2,22]	Depends on deployment	Not their target task	No	Yes, on vision/CL benchmarks

This proposal	Yes	Yes (by design)	Yes (Sec. 6)	No – not yet built
---------------	-----	-----------------	--------------	--------------------

9.4 Periodic Embedding Recomputation Cost (new, Revision 4)

Section 4.5’s spectral embedding is a fundamentally different cost profile from everything else in this section: a periodic, batch computation over the whole graph, not a per-keystroke one, so it does not belong folded into Section 9.2’s latency estimate. Computing the exact top eigenvectors of an $|V| \times |V|$ Laplacian directly is infeasible at the scale assumed in Section 9.1 ($|V| \approx 5 \times 10^4$): direct diagonalization costs $O(|V|^3)$, on the order of 10^{14} operations, far beyond any on-device budget. In practice, only the two leading eigenvectors are needed (Eq. (37)), which iterative methods (Lanczos iteration, or power iteration on a shifted/inverted operator) can extract directly from the graph’s sparse structure without ever forming the dense matrix: roughly $O(k \cdot \text{nnz}(L) \cdot \text{iterations})$, where $k = 2$, $\text{nnz}(L) \approx |V| \times d \approx 5 \times 10^4 \times 20 \approx 10^6$ non-zero entries, and iteration counts for the top two eigenvectors of a graph Laplacian of this size typically number in the tens to low hundreds depending on the spectral gap between λ_2 and λ_3 (a gap this paper has not estimated for a real personal-typing graph). That suggests on the order of 10^8 - 10^9 elementary operations per recomputation — several orders of magnitude more than a single Eq. (6) local-push call, but, as a *periodic* rather than per-keystroke cost, plausibly still sub-second on current mobile CPU hardware. This is a back-of-envelope estimate under the same caveats as the rest of this section: not measured, and worst-case behavior on a real, heavy-tailed personal graph could differ materially from this typical-case reasoning.

Two gaps Section 4.5.1 already named become concrete costs here rather than abstract ones. First, T_{embed} — how often this recomputation runs — is unspecified by the addendum and unspecified here; it trades off staleness against cost, and neither this paper nor the addendum picks a value. Second, and more operationally pressing: a node that enters the graph after the most recent recomputation has no coordinate (x_u, y_u) at all until the next one runs, and nothing in Section 4.5 or this integration specifies what position(i) (Eq. (38)) should do with such a node in the interim — omit it from the weighted centroid, treat it as sitting at the origin, or something else. This is a genuine implementation gap, not resolved here by design rather than by oversight: inventing a specific interpolation rule on the addendum’s behalf would be presenting a choice as settled that neither the addendum nor this paper has actually made. Section 12 lists both gaps.

10. Proposed Evaluation Protocol

This section specifies the experiment that would need to be run to support or refute this paper’s central claims. No part of it has been executed. We give it in enough detail that it could be implemented directly, because a vague gesture at “future evaluation” would leave the paper’s

strongest claims untestable in practice, which we think is a worse failure than admitting plainly that the work is not yet done.

10.1 Data

Because the entire premise of the system is that it trains on a single person’s private typing history, there is no existing public dataset that matches its intended deployment setting exactly. We propose two complementary data sources:

1. Synthetic typing histories, generated by sampling from a mixture of stable topic clusters (representing long-term habits) and injected transient bursts (representing situational patterns, including explicit valence-conflict bursts of the kind described in Section 5), so that ground truth for “this should have been suppressed” and “this should have been promoted” is known by construction. This lets the valence-conflict metric (below) be evaluated even before any real user data is available. The same generator is extended, for the necessity-gating evaluation of Section 7, to also inject a controlled proportion of low-relevance distractor candidates — tokens sampled near, but not from, the seed context’s true topic cluster — each carrying a held-out low-relevance label, so that Hypothesis 5 (Sec. 10.4) has ground truth to score against without requiring additional real-user annotation.
2. Real personal corpora, collected with informed consent, from a small number of volunteer participants’ own message and note history, used strictly on-device or in a sandboxed offline analysis with the participant’s own data, never pooled across participants. This is necessary because synthetic data cannot fully capture the irregularity of real valence-conflict language use illustrated in Section 5.

Extension (Revision 3). The same synthetic generator is extended further, for Section 6.9’s evaluation, to inject controlled sustained-arousal bursts (sequences of elevated, self-relative typing-rate deviation persisting across several consecutive keystrokes, with ground-truth streak boundaries known by construction, Eq. (27)), controlled shock-retreat trigger sequences (a sharp valence drop co-occurring with a correction burst, per Eq. (30), each carrying a held-out label marking whether it was constructed as a genuine shock or as an ordinary frustrated-typing false-positive case), and controlled settling sequences (an active-prey-set burst followed either by a confirmed selection or by a lapse past T_{grace} with no further activity), so that Hypothesis 6 (Sec. 10.4) and its ablations have ground truth to score against without requiring real-user annotation of emotional state, which this paper has no reliable way to collect in the first place.

Extension (Revision 4). Hypotheses 7 and 8 below need no new synthetic construction beyond what Sections 10.1’s existing sources already provide: $\varphi(u)$ (Eq. (18)) and the spectral embedding coordinates (Eq. (37)) are both computed directly from whatever graph B4/B5/B6 has already built from either data source above, and reconvergence sequences are simply pairs of constructed sentences that diverge and later share a

token — a property of the input text, not a new labeled construct requiring its own generator extension. The one new requirement is incidental rather than architectural: enough distinct nodes with a meaningful spread of $\varphi(u)$ values to make a correlation test (Hypothesis 7) statistically meaningful, which the synthetic generator’s existing mixture of stable and transient clusters already tends to produce as a side effect, without modification.

10.2 Baselines

- B1 - Kneser-Ney smoothed trigram model [17]: the standard, strong, low-resource baseline.
- B2 - Unsigned, function-typed diffusion (ablation): the architecture of Sections 3-4 without the valence extension of Section 6, to isolate what the base function-typed diffusion idea contributes on its own before adding valence gating. (*Status note, Revision 2: renamed from “unsigned single-channel diffusion”; the ablation’s role — diffusion over transmission-type edges alone, no signed layer — is unchanged, only the underlying edge-typing mechanism it inherits from Section 3 is different.*)
- B3 - Adapted dual-memory continual-learning baseline: a reimplementation of the fast/slow buffer idea from CLS-ER [2] or ITDMS [22], adapted from their original image/task-incremental setting to next-token prediction over the same personal token stream. (*Status note, Revision 2: this baseline is no longer a close cousin of this paper’s own architecture, since Section 3.2 no longer shares its consolidation mechanism; it is now a genuine outside-literature comparison point, which the addendum incorporated in this revision argues is a more meaningful test of Hypothesis 3 below, not a weaker one, precisely because there is no shared ancestry left to narrow the gap artificially.*)
- B4 - Full proposed system: Sections 3-6 in full, including silk-type functional differentiation, fan-dispersion seeding, the signed diffusion read-out, and the third, suppressive set.
- B5 - B4 with residue-priority arbitration: the full B4 system, augmented with the residue-priority arbitration rule of Section 6.8, used specifically to isolate the contribution of that rule in Hypothesis 4 below.
- B0 - No-gating ablation (necessity-gating baseline): always surface B4’s top-1 candidate regardless of confidence, i.e., B4 with the necessity gate of Section 7 (Algorithm 1, steps 12-14) removed entirely. We keep the label B0 rather than introducing a “B6,” to signal that this is not a sixth competing architecture alongside B1-B5 but a null ablation specifically paired with the gating mechanism of Section 7, used only in Hypothesis 5 below.

Status note (Revision 3). The B6 label this note deliberately declined to claim is used below, for exactly the case it was reserved for: a genuine sixth architecture variant, not a null ablation.

- B6 - B5 augmented with the emotional engine (new): the full B5 system (Sections 3-6, including residue-priority arbitration), further

augmented with Section 6.9’s compounding cluster reinforcement, growing engagement radius, shock-retreat, and grace-period settling, used to isolate the emotional engine’s overall contribution in Hypothesis 6 below. Sections 6.9.6-6.9.7’s emergent sub-state clustering and nonlinear interaction weight are held out of B6 by default, since Section 6.9.7 itself leaves their relationship to the rest of Section 6.9 an open integration question rather than a specified substitution; a variant of B6 including them is left for future work rather than folded into this protocol.

Status note (Revision 4). Section 4.5’s geometric trajectories introduce no B7. Every other baseline and ablation in this protocol isolates a difference in system *behavior* — what gets predicted, reinforced, suppressed, or gated. Section 4.5 changes none of these; $\varphi(u)$ and the embedding of Eq. (37) are computed identically regardless of which of B1-B6 produced the underlying graph. Hypotheses 7 and 8 below are evaluated directly against whichever of B4, B5, or B6’s graphs is already being built for the other hypotheses, not against a new architecture built specifically to test them.

10.3 Metrics

- **Overall top-3 accuracy:** standard next-token prediction accuracy across a held-out 10% of the typing stream.
- **Cold-start top-3 accuracy:** accuracy restricted to sequences whose target token appears for the first time in the held-out portion, but whose graph neighbors already have transmission-type edges (Section 3.3) - directly operationalizing the falsifiable test proposed in the original working notes this paper formalizes.
- **Valence-conflict accuracy:** accuracy, and separately, suppression correctness (whether a rejected association is down-ranked rather than up-ranked on the next relevant context), evaluated specifically on the constructed valence-conflict subset described above.
- **Unhelpful-suggestion rate:** the fraction of keystrokes at which a suggestion is shown despite the shown candidate carrying a held-out low-relevance label (Sec. 10.1). The necessity-gating hypothesis (Hypothesis 5, Sec. 10.4) is evaluated on the reduction of this rate.
- **Sustained-engagement accuracy (Revision 3):** top-3 accuracy restricted to keystroke sequences falling inside a constructed sustained-arousal burst (Sec. 10.1), evaluating whether compounding reinforcement and growing radius (Sections 6.9.2-6.9.3) measurably help on exactly the cases they target.
- **Shock-suppression correctness (Revision 3):** whether an edge implicated in a constructed shock-retreat sequence (Sec. 10.1) is correctly moved into *Esuppressed* (Eq. (31)), reported alongside its false-positive rate on ordinary frustrated-but-non-shock typing, using the held-out genuine/false-positive labels described in Section 10.1.
- **Centrality-dispersion correlation (Revision 4):** the Spearman rank correlation between $\varphi(u)$ (Eq. (18)) and $\|(x_u, y_u) - \text{centroid}\|$ (Eq. (37)), computed across all nodes in a representative graph, with signifi-

cance assessed against the null of no correlation. This is a structural property of the embedding, not a comparison between architectures, and is reported once per graph rather than per candidate or per keystroke.

- **Reconvergence-gap ratio (Revision 4):** for each constructed pair of trajectories that diverge and later share a token (Sec. 10.1), the ratio of $\|\text{gap}(i)\|$ (Eq. (43)) immediately after the shared token to $\|\text{gap}(i)\|$ immediately before it, computed separately for shared tokens with low $\varphi(u)$ and shared tokens with high $\varphi(u)$, so the two conditions can be compared directly rather than pooled into one average that would obscure the difference Section 4.5.5 predicts between them.

10.4 Hypotheses and Pass/Fail Criteria

Hypothesis 1. B4 outperforms B1 on cold-start accuracy by a margin larger than on overall accuracy, with the gap significant under a paired bootstrap test at $p < 0.05$. Failure of this hypothesis would mean the core diffusion idea (Sections 3-4) is not earning its complexity over a plain smoothed n-gram model, independent of the valence question.

Hypothesis 2. B4 outperforms B2 on valence-conflict suppression correctness by a margin significant under McNemar’s test at $p < 0.05$, while showing no significant regression on overall accuracy. Failure of this hypothesis would mean the valence-gated extension of Section 6 is not solving the problem it was designed to solve, and the added complexity of signed edges and a third channel is not justified.

Hypothesis 3. B4 performs comparably to or better than B3 on cold-start and valence-conflict accuracy. Failure of this hypothesis would mean the adjacent, already-published dual-memory continual-learning literature (Section 2.2), adapted to this task, already solves the problem this paper targets, which would substantially weaken the novelty claim in Section 11 regardless of how Hypotheses 1 and 2 resolve.

We consider Hypothesis 3 the single most important test in this protocol. (*Status note, Revision 2: under Revision 1, this was framed as the test most likely to fail or come out mixed precisely because B3 sat close to this paper’s own mechanism in the literature of Section 2.2. That is no longer the reason it matters most. With Section 3.2’s mechanism no longer sharing B3’s lineage, a pass here is a stronger result than it used to be — there is no shared ancestry left to narrow the gap — but a failure would be equally informative in the other direction: it would mean the specific, narrower novelty this revision now claims does not translate into a measurable advantage over the nearest adjacent published architecture, with no fallback claim to retreat to, per Section 11.1.*)

Hypothesis 4. B5 improves suppression correctness or overall accuracy over B4 without significantly regressing cold-start accuracy specifically, tested via a paired bootstrap comparing B4 with and without the residue-priority arbitration rule of Section 6.8 active, at $p < 0.05$. Failure of this hypothesis in the specific form of a cold-start regression, rather than simply no improvement, would confirm the risk named in Section 6.8.6: that residue-priority arbitration actively works against the architecture’s own founding goal, not merely that it fails to help. This dis-

inction - no effect versus active harm to cold-start cases - should be reported separately, not collapsed into a single pass/fail number.

Hypothesis 5. Necessity-gated suggestion (B4 augmented with the gate of Section 7) reduces the unhelpful-suggestion rate (Sec. 10.3) relative to B0 (Sec. 10.2), without a significant drop in top-3 accuracy on the cases where a suggestion is still shown, tested via a paired bootstrap at $p < 0.05$, consistent with the other hypotheses above. Failure of this hypothesis would mean the gate of Section 7 adds a silent-keyboard cost without measurably improving suggestion quality, and B0 - showing every top-1 candidate regardless of confidence - remains the simpler, better design. Unlike Hypotheses 1-4, this hypothesis concerns a question orthogonal to ranking quality: *whether* a candidate should be shown at all, not *which* candidate would be shown; a failure here says nothing about the validity of Sections 3-6, only about Section 7.

Hypothesis 6 (Revision 3). B6 outperforms B5 on sustained-engagement accuracy (Sec. 10.3) without a significant regression on overall or cold-start accuracy, tested via a paired bootstrap at $p < 0.05$; shock-suppression correctness and its false-positive rate on ordinary frustrated typing are reported separately, following the same no-effect-versus-active-harm distinction Hypothesis 4 already insists on, rather than collapsed into a single pass/fail number. Failure of the accuracy comparison would mean Section 6.9's mechanisms, taken together, do not measurably help on exactly the excited-typing cases they were built for, independent of how well Sections 3-7 perform otherwise; a poor false-positive rate on shock detection specifically would mean Eq. (30) is not isolating the intended class of events (Section 6.9.4's own stated concern), regardless of how the accuracy comparison resolves. Given how many distinct mechanisms Section 6.9 bundles into B6, ablations (h)-(k) below are what actually attribute a Hypothesis 6 result — positive or negative — to a specific mechanism, rather than to the cluster as a whole.

Hypothesis 7 (Revision 4). The centrality-dispersion correlation (Sec. 10.3) is negative and significant (a Spearman ρ meaningfully below zero, $p < 0.05$: high $\varphi(u)$ predicts *small* distance from centroid, low $\varphi(u)$ predicts large distance) across a representative graph. Unlike Hypotheses 1-6, this is not a comparison between architectures; it is a structural claim about a single graph's geometry, true or false regardless of which baseline built it. Failure would mean Section 4.5.1's central hypothesis does not hold, and any interpretability claim resting on "off-center nodes are the narrow, specific ones" (including the intuitive reading of the worked example in Section 4.5.9) would be unfounded, though Sections 4.5.2-4.5.4's trajectory and gap constructions would remain well-defined regardless, since nothing about them requires this correlation to be true.

Hypothesis 8 (Revision 4). The reconvergence-gap ratio (Sec. 10.3) is significantly smaller for shared low- φ tokens than for shared high- φ tokens, tested via a paired comparison across constructed diverge-then-reconverge sequences at $p < 0.05$. Failure would mean Section 4.5.5's central claim — that reconvergence tightness tracks the shared token's own dispersion — does not hold, independent of whether Hypothesis 7

holds; the two are related but logically separate claims, and this protocol deliberately does not let a pass on one substitute for evidence on the other.

10.5 Ablations

At minimum: (a) removing the suppressed set $E_{\text{suppressed}}$ while keeping signed, function-typed edges, to isolate the third set’s specific contribution flagged as speculative in Section 6.4; (b) replacing the frequency-derived (typing-cadence) valence scorer of Section 6.2 with a random or constant-zero valence signal, to confirm that any gains attributed to valence gating are not actually coming from some other side effect of the signed-edge formalism; (c) sweeping θ_{TF} , η_{TF} , τ_V , α , ϵ to characterize sensitivity, addressing the parameter-setting question left open throughout Sections 3-6 (T and the old promotion threshold θ no longer apply, Section 3.3); (d) sweeping ρ (Section 6.8) over $\{0, 0.25, 0.5, 0.75, 1.0\}$ and reporting cold-start accuracy and suppression correctness at each value, given the finding in Section 6.8 that behavior near $\rho = 0.5$ is not representative of a “balanced” setting; (e) sweeping the necessity threshold κ_N (Sec. 7.3) over a small grid, and separately comparing a fixed global κ_N against the per-user adaptive variant flagged in Sec. 7.2’s Modify/Magnify step, to characterize how much of any Hypothesis 5 effect depends on per-user adaptation specifically versus a single global threshold.

Two further ablations were added in Revision 2 for constructions new at that point:

- (f) **Reversibility ablation.** Replacing the freely-reversible TF-based role assignment of Eq. (17) with a one-directional, EWC-style variant — an edge that crosses θ_{TF} is permanently transmission-type regardless of later TF values — to isolate whether the “no-ratchet” property flagged in Section 3.3 as one of two substantive differences from the mechanism it replaces contributes measurably to any of Hypotheses 1-3, or is a difference without a measurable effect at the timescales this evaluation covers. A null result here would not invalidate Section 3.2’s motivation but would narrow its practical importance considerably.
- (g) **Seed-construction ablation.** Replacing the fan-dispersion-weighted, multi-token seed of Eq. (19) with the single-token indicator seed it replaces, holding everything else in B4 fixed, to isolate the specific contribution of Section 4.1.1 independent of silk-type functional differentiation’s contribution to Hypotheses 1-3. This directly tests whether $\varphi(u)$ -weighted dispersion discounting is pulling its own weight, or whether the gains (if any) found in Hypotheses 1-3 are attributable entirely to transmission-fidelity typing.

Four further ablations are added in this revision for Section 6.9, each removing exactly one mechanism from B6 while holding the rest of it fixed, so that a Hypothesis 6 result can be attributed to a specific piece rather than to the bundle as a whole:

- (h) **Compounding-reinforcement ablation.** Removing Eqs. (25)-(26)’s compounding budget from B6 while keeping growing radius, shock-retreat, and settling active, to isolate whether compounding reinforcement specifically drives any Hypothesis 6 effect, or is inert — or actively harmful — on its own.
- (i) **Growing-radius ablation.** Removing the $\alpha_{effective}$ substitution of Eq. (29) from B6 (running local push at the fixed α_{base} throughout), to isolate growing radius’s specific contribution, and to give a direct, measured comparison against the cost-inflation risk flagged in Sections 6.9.3, 4.3, and 9.2 — a positive accuracy effect here would need to be weighed against a measured, not merely extrapolated, latency cost.
- (j) **Shock-retreat ablation.** Removing Eqs. (30)-(31)’s detection and local-suppression path and the $\kappa_N(t)$ modulation of Eq. (36) from B6, falling back to ordinary integral-triggered suppression (Eq. (21)) and a constant κ_N , to isolate whether the discontinuous shock path contributes anything beyond what gradual, valence-integral-driven suppression already provides.
- (k) **Settling ablation.** Disabling the grace-period rollback of Eq. (32) — letting compounding-reinforcement and radius-growth deltas decay at their ordinary background rates regardless of whether a streak was confirmed or lapsed — to isolate whether distinguishing confirmed from unconfirmed excited streaks specifically improves calibration, or whether ordinary decay already handles it adequately.

Status note (Revision 4). No ablation (l) is added for Section 4.5. An ablation isolates a mechanism’s contribution to behavior by removing it and measuring the difference; Section 4.5 contributes no behavior to remove. Hypotheses 7 and 8 are tested by computing $\phi(u)$, the embedding, and reconvergence ratios directly — the same way a reported statistic is checked, not the way an architectural component is isolated.

11. Discussion: Novelty, Non-Obviousness, and Patentability

We include this discussion because a reasonable reader of a document like this will ask the question directly, and because answering it honestly is more useful than either ignoring it or asserting a conclusion the related-work section does not support. This section is not legal advice. We are not patent attorneys, and nothing here substitutes for a formal freedom-to-operate search or professional legal opinion before any filing.

11.1 The Trade This Revision Makes

New in Revision 2. Revision 1’s version of this section worked by conceding component 1 below (the two-channel memory structure) as not novel, and staking the paper’s claim on the narrower components built on top of it. That structure of argument depended on component 1 being a real, load-bearing part of the architecture that the paper could

honestly concede — a well-established fallback sitting underneath the narrower claim, even if the narrower claim itself was never advanced. Section 3.2 removes component 1 outright and replaces it with silk-type functional differentiation. This is not merely swapping one component for another of the same kind: it removes the one piece of this paper’s architecture that rested on thirty-five years of published, cross-validated prior work (Section 2.2), and replaces it with a mechanism that is, by the addendum’s own account, new and untested at every level. The result is a *broader* potential novelty claim — nothing in the new core mechanism concedes ground the way the old Section 3 did — purchased at the cost of a *narrower* margin for error: if Sections 3.2-3.3 as now written do not hold up empirically (Section 10, Hypotheses 1 and 3 especially), there is no longer a fallback position of “well, at least the memory-consolidation half was known to work.” We think this trade is worth stating in exactly these terms rather than letting the broader claim below read as a strict improvement over Revision 1’s.

11.2 Components

With that stated, novelty and non-obviousness are ordinarily assessed against components, and it is worth being precise about which of them this paper’s contribution actually rests on:

1. Silk-type functional differentiation (Section 3.2) and the transmission/capture type role assignment built on it (Section 3.3). This replaces the retired two-channel, dual-timescale memory structure, which Section 2.2 documented as clearly *not* novel (DualNet [21], CLS-ER [2], and ITDMS [22] all use versions of it). The replacement is, by the same search, not found in that literature either — no reviewed architecture types an edge’s participation in propagation by an ongoing, freely-reversible measure of predictive success, as opposed to elapsed time, frequency, or a one-directional consolidation clock. Whether that is because it is genuinely novel or because it has simply not been tried (perhaps for reasons this paper has not anticipated) is precisely the distinction Section 11.1 above says this revision cannot yet tell apart with confidence, since the mechanism has not been built. This is now the single most load-bearing component in the paper’s novelty case, which is a stronger position only if Hypotheses 1 and 3 (Section 10.4) actually pass.
2. Diffusion-based (rather than gradient- or replay-based) read-out on a live, per-user, keystroke-level graph, seeded by dispersion-weighted recent tokens rather than a single indicator (Sections 4, 4.1.1). This claim is structurally the same shape as Revision 1’s component 2, extended to cover the fan-dispersion seed construction: personalized-PageRank-style diffusion, computed anew via local push at each keystroke, over a graph typed by the mechanism in component 1, seeded by a construction (Eq. (19)) that discounts high-entropy, low-informativeness recent tokens rather than treating the single most recent token as the only anchor. Section 2 did not find this specific combination in the literature searched. Absence of evidence in a search is not proof of ab-

sence in the world; a genuine freedom-to-operate search would need to be broader and conducted by someone trained to do it.

3. The valence-gated, signed extension of Section 6, now with a behavioral rather than lexical valence signal. This remains, in our assessment, the strongest candidate for a genuine non-obviousness argument, for the same structural reason as in Revision 1: Section 5 shows the naive combination does not work as-is (Proposition 1, now restated against Eqs. (1), (16)-(17) and holding, if anything, more cleanly than before), and the fix requires a nontrivial extension — signed edges, a suppression rule now stated independently of the retired promotion mechanism, a third memory set whose interaction with transmission-fidelity typing raises a new stability question this revision surfaces rather than resolves (Section 6.4), and an unresolved theoretical gap in the local-push convergence argument that Section 3.3 shows is now a *compound* gap rather than a single one (Section 6.7). One piece of this component’s grounding has shifted, though: the valence signal itself no longer rests on a validated sentiment-analysis tool (VADER [12]) adapted to a new setting, but on a keystroke-dynamics signal [27] repurposed for a role (valence proxy) that field was never built for, motivated by a biological analogy [26] rather than by that field’s own literature. This is a different, and arguably thinner, form of borrowed credibility than Revision 1’s version of this component had, and we think it is more honest to name that shift than to let the “strongest candidate” framing above imply the ground under it is unchanged.
4. The necessity-gating mechanism of Section 7. Unaffected in substance by this revision beyond the $\text{val} \rightarrow \text{val}'$ substitution in Eq. (15)’s *valence_{urgency}* term (Section 6.2). Confidence gating and abstention are well-established general patterns; a claim of novelty cannot rest on the general pattern, only on the specific combination (entropy confidence, valence urgency, exhaustion) via Eq. (15), which Section 2’s search did not locate applied to keystroke-level suggestion gating. This remains, by construction, the weakest of the four components: no structural proof analogous to Proposition 1 forces the gate’s specific form, the mechanism was generated via a creativity checklist (SCAMPER, Sec. 7.2) rather than derived from a demonstrated failure mode, and its non-obviousness case rests almost entirely on Hypothesis 5 (Sec. 10.4) actually passing.
5. The emotional engine of Section 6.9 — compounding reinforcement, growing radius, shock-retreat, settling, sub-state clustering, and the nonlinear interaction weight (Revision 3). This is, by a wide margin, the weakest of the five components, and unlike component 4 above, it is weaker by the addendum’s own explicit design rather than only by our retrospective assessment: Section 0 of the addendum draws its own line between the two established citations underlying Sections 6.9.1 and 6.9.8 and the remaining five mechanisms, which it identifies as analogy-derived invention rather than adaptation of any demonstrated pattern. Confidence gating (component 4) at least combines established decision-theoretic building blocks; compounding

reinforcement, growing radius, shock-retreat, settling, and sub-state clustering have no comparably established general pattern to lean on at all — they exist because a single motivating analogy was followed through its logical implications, a generation method with less structure behind it than even SCAMPER’s checklist (Section 7.2). Its non-obviousness case, such as it is, rests entirely on Hypothesis 6 and its four accompanying ablations (Sections 10.4–10.5) actually passing, and on considerably more new free parameters (Section 12) than any other single component in this paper earning their keep, rather than merely fitting whatever data they are tuned against.

6. The geometric trajectories of Section 4.5 — a live spectral embedding, per-sentence trajectories, and the gap-as-shape construction between two of them (Revision 4). This component is not weaker or stronger than components 1–5 so much as a different *kind* of claim, and the patentability question for it should be framed accordingly. Components 1–5 are, in different ways, claims about a predictive or decision mechanism: what gets typed as an edge, how diffusion is seeded, when a suggestion surfaces. Component 6 predicts and decides nothing; the underlying spectral-embedding technique itself is old and unpatentable on its own terms (Section 2.9), so any non-obviousness argument here would need to rest on the specific combination — a live, per-user, keystroke-evolving graph read out as a two-dimensional trajectory, with a defined notion of the shape between two such trajectories — as a method of representing and analyzing system state, the kind of claim closer in spirit to a data-visualization or diagnostic-instrumentation patent than to a prediction-architecture one. Section 2.9’s own search was not conducted with this framing in mind and should not be read as having ruled out prior art of that specific kind.

11.3 Taken Together

We think components 2 and 3 constitute a coherent, honestly-scoped novelty claim on their own, largely independent of how component 1 resolves empirically, since diffusion-based read-out and the signed extension do not logically depend on silk-type differentiation being the specific typing mechanism underneath them — a reader unconvinced by component 1 could still find 2 and 3 worth taking seriously against a more conventional edge-weighting scheme. Component 1 is the higher-variance addition: potentially the broadest claim in the paper if Hypotheses 1 and 3 pass, and the component with the least established literature underneath it if they do not, for the reasons stated in Section 11.1. Component 4 remains a more provisional, weaker addition to whichever of the above holds up, conditional on (a) a broader prior-art search than the one conducted for this paper, and (b) the empirical validation specified in Section 10 actually being carried out. A mechanism that does not outperform baseline B3 in practice (Hypothesis 3) would have working code but a much weaker claim to inventive step regardless of how novel its construction looks on paper — and, per Section 11.1, this revision has removed the option of falling back on component 1’s predecessor being

separately well-established if that happens. We would not advise treating this paper, on its own, as sufficient grounds for a patent filing; we would advise treating it as the technical specification a patent attorney and the evaluation in Section 10 would both need as inputs.

Extension (Revision 3). Component 5 sits outside the hierarchy above rather than inside it: nothing in components 1-4's own novelty argument depends structurally on Section 6.9, and Section 6.9 could, in principle, be removed entirely without touching components 1-4 (Section 6's editorial note describes it as additive rather than load-bearing for exactly this reason). We would not advise weighting component 5 meaningfully in any patentability assessment until Hypothesis 6 has actually been run. Unlike components 1-4, we do not think it currently clears even the lower bar those four components clear — a reasonable non-obviousness argument that empirical failure would weaken rather than invalidate. Empirical failure of Hypothesis 6 would remove essentially the entire basis for treating component 5 as anything more than an untested idea, since it has no comparable fallback (no thirty-five-year memory-consolidation lineage as component 1 once had, no structural impossibility proof as component 3 has in Proposition 1, no established decision-theoretic pattern as component 4 at least partially has) to retreat to if the empirical case does not hold up.

Extension (Revision 4). Component 6 sits outside the hierarchy in a third, distinct way, neither load-bearing like components 1-4 nor a weaker instance of the same kind of claim like component 5. It could be removed entirely without touching any other component's argument, exactly as component 5 could — but unlike component 5, its empirical failure would not weaken this paper's predictive claims at all, because it makes none: Hypotheses 7 and 8 failing would mean the embedding does not track connectivity structure the way Section 4.5.1 hypothesizes, which says nothing about whether Sections 3-7 correctly predict or gate suggestions. We would not advise including component 6 in a patentability assessment framed around prediction quality, and would advise, if it is pursued at all, framing it separately as an interpretability or diagnostic-tooling contribution assessed on its own terms (Section 2.9), against a different, and to our knowledge not yet searched, body of prior art.

12. Ethical Considerations

A personal intent graph of the kind described here is, by construction, a structured, compressed record of what a person cares about, worries about, and does repeatedly — plausibly including health conditions, relationships, finances, and situational vulnerabilities, inferable from the shape of the graph even without reading any raw text. Keeping that graph on-device and never transmitting it (Section 8) removes the most obvious risk — a server-side breach exposing many users' data at once — but does not remove device-level risk: a lost or compromised phone, a poorly secured local backup, or a future product decision to add cloud

sync for cross-device continuity would each reintroduce exposure that the current design avoids. Any real implementation should treat the graph as sensitive at rest by default (encrypted, excluded from routine cloud backups unless the user opts in explicitly and understands the tradeoff), and should give users a clear, low-friction way to inspect and delete it, including the suppressed set proposed in Section 6.4, which is specifically designed to retain memory of things the user tried to reject - exactly the category of data a user would most want to be able to purge completely and verifiably.

New consideration (Revision 2). Section 8 already names the specific new risk Revision 2 introduces: the per-user typing-cadence baseline (Δt_{ref} and related statistics, Section 6.2) is a behavioral biometric, not merely a derived preference score. The ethical framing of “give users a clear way to inspect and delete” stated above should extend explicitly to this data, not only to the graph — a user asking “what does this keyboard know about me” has a reasonable expectation that the answer covers how they type, not only what they type about, and an implementation that lets someone delete their word-association graph while quietly retaining a rolling biometric typing profile would not meet the spirit of the recommendation above even if it met the letter of it.

New consideration (Revision 3). Section 6.9.6’s emergent sub-state clustering raises the same question Revision 2 raised, one level up. Where Δt_{ref} and the arousal baseline of Section 6.9.1 are single behavioral-biometric scalars, a discovered set of per-user typing sub-states is closer to a rudimentary emotional profile — an inventory of however many distinguishable excited states a specific person’s own typing supports, discovered without their explicit input. A user asking “what does this keyboard know about me” arguably has an even stronger interest in knowing that such a profile exists and in being able to inspect or delete it than in knowing about a single cadence baseline, precisely because a small set of named or numbered clusters is more legible, and potentially more revealing if exposed, than a raw timing statistic would be on its own. Any real implementation should extend the inspect-and-delete recommendation above to cover discovered sub-state clusters explicitly, not fold them silently into “the graph” or “the biometric baseline” as though they were the same kind of thing.

New consideration (Revision 4). Section 4.5 does not add a new category of sensitive data — position(i) and gap(i) are computed entirely from the graph and diffusion outputs already covered by the considerations above — but it does add a new *representation* of that data worth naming on its own terms: a literal, visualizable picture of a person’s typing as it moves through their own intent graph. If a real implementation ever surfaces this — for debugging, or as a “here’s what your keyboard has learned” feature — it would be an unusually legible and potentially unsettling artifact for a user to encounter, considerably more vivid than a raw graph export or a cadence statistic, in the same way a discovered sub-state cluster (above) is more legible than the timing scalar it is built from. The same inspect-and-delete rights already recommended for the graph, the cadence baseline, and the sub-state clusters should extend to

any stored or cached trajectory data without needing a separate justification for why this representation is different: it is the same underlying data, made easier for a human to read at a glance, which is a reason for *more* care in how it is exposed, not less.

Section 7’s motivation for necessity gating - that a keyboard which always surfaces its top-ranked candidate, even weakly supported, risks eroding user trust - is as much an ethical design consideration as a UX one: a system that trains users to ignore its suggestions by being frequently wrong is arguably worse than a system that is visibly, honestly silent. Separately, Appendix A records that an early brainstorming direction for this project - using accumulated graph structure to infer a user’s gender or other protected attributes - was deliberately not pursued as a feature and was redirected toward the attribute-privacy question already listed as future work in Section 8; we surface that provenance here for transparency rather than leaving it implicit.

13. Conclusion

We have formalized a function-typed, diffusion-based architecture for on-device predictive text — one that, as of this revision, types each edge by an ongoing, freely-reversible measure of transmission fidelity rather than by a consolidation clock, and seeds diffusion from a dispersion-weighted combination of recent tokens rather than a single one — located its novelty precisely against a crowded adjacent literature rather than overstating it, proved a specific structural limitation of the base design, proposed a signed, valence-gated extension (now with a behavioral rather than lexical valence signal) to address that limitation, and proposed a separate, more speculative necessity-gating mechanism for deciding whether to surface a suggestion at all - while being explicit, throughout, about which parts of each piece are established and which are new and unverified. We have also been explicit, in this revision specifically, about what replacing the earlier two-channel memory structure costs the paper’s own argument: a broader novelty claim with no established fallback underneath it (Section 11.1), rather than a strict improvement over the version it replaces. We then added a second, orthogonal affective axis - arousal - on top of the existing valence signal, and a cluster of six further mechanisms built on top of *that*, motivated by an attack-and-struggle analogy rather than any additional cited finding beyond the two ([28], [29]) grounding the axis itself and its cross-channel caveat, and said, at every point of introduction rather than only in retrospect, that this cluster is the least-grounded and most parameter-heavy component in the paper. Most recently, we have added a component of a genuinely different kind from everything before it: a live spectral embedding of the same graph (Section 4.5), turning each sentence into a literal trajectory and the divergence between two of them into a measurable shape, proposed purely for interpretability and for testing two specific structural hypotheses about the embedding’s geometry - not as an input to any prediction, reinforcement, or gating decision anywhere

in this paper, a distinction we have tried to keep sharp rather than let the section's more vivid, visual character blur it. Integrating it also surfaced a genuine interaction between two separately-authored addenda that neither could have caught alone: Section 4.5's shared-prefix proof, sound when written, needed a stated qualification once Section 6.9's arousal-driven diffusion parameter made timing, not just token content, part of what "identical input" means. We have deliberately not reported experimental results, because none exist yet. The honest next step, for this specific project, is exactly the one named in Section 10: build the systems described there (B0 through B6, plus the ablations of Section 10.5), run the eight hypotheses, and let Hypotheses 1 and 3 in particular - whether the function-typed core actually beats a plain n-gram baseline on cold-start, and whether it beats the closest existing published architecture at all - decide how strong the broadened novelty claim of Section 11 actually turns out to be, now that it has no fallback claim to retreat to if they fail. Hypothesis 5, on necessity gating, Hypothesis 6, on the emotional engine, and Hypotheses 7-8, on the geometric embedding, remain separate and progressively lower-stakes tests: a failure at any of them would narrow the paper's contribution without undermining Sections 3-6, and a failure specifically at Hypotheses 7-8 would not touch the architecture's predictive claims at all, only the embedding's own value as an interpretability tool, since Section 11.2 already treats component 6 on its own terms rather than folding it into the architecture's novelty case by default. Everything else in this paper is a specification for that experiment, not a substitute for it.

References

- [1] R. Andersen, F. Chung, and K. Lang. Local graph partitioning using PageRank vectors. In *47th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2006)*, pages 475-486, 2006.
- [2] E. Arani, F. Sarfraz, and B. Zonooz. Learning fast, learning slow: A general continual learning method based on complementary learning system. In *International Conference on Learning Representations (ICLR)*, 2022.
- [3] D. Cartwright and F. Harary. Structural balance: A generalization of Heider's theory. *Psychological Review*, 63(5):277-293, 1956.
- [4] F. Chung. The heat kernel as the PageRank of a graph. *Proceedings of the National Academy of Sciences*, 104(50):19735-19740, 2007.
- [5] M. Dorigo. *Optimization, Learning and Natural Algorithms*. PhD thesis, Politecnico di Milano, 1992.
- [6] C. Dwork. Differential privacy. In *33rd International Colloquium on Automata, Languages and Programming (ICALP)*, pages 1-12, 2006.
- [7] C. Genest, S. Weerahandi, and J. V. Zidek. Aggregating opinions through logarithmic pooling. *Theory and Decision*, 17(1):61-70, 1984.
- [8] C. Genest and J. V. Zidek. Combining probability distributions: A critique and an annotated bibliography. *Statistical Science*, 1(1):114-135, 1986.

[9] F. Glover. Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research*, 13(5):533-549, 1986.

[10] A. Hard, K. Rao, R. Mathews, S. Ramaswamy, F. Beaufays, S. Augenstein, H. Eichner, C. Kiddon, and D. Ramage. Federated learning for mobile keyboard prediction. *arXiv preprint arXiv:1811.03604*, 2018.

[11] G. E. Hinton and D. C. Plaut. Using fast weights to deblur old memories. In *Proceedings of the Ninth Annual Conference of the Cognitive Science Society*, pages 177-186. Erlbaum, 1987.

[12] C. J. Hutto and E. Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Eighth International AAAI Conference on Weblogs and Social Media (ICWSM)*, 2014.

[13] G. Jeh and J. Widom. Scaling personalized web search. In *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pages 271-279, 2003.

[14] S. Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 35(3):400-401, 1987.

[15] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521-3526, 2017.

[16] J. Klicpera, A. Bojchevski, and S. Günnemann. Predict then propagate: Graph neural networks meet personalized PageRank. In *International Conference on Learning Representations (ICLR)*, 2019.

[17] R. Kneser and H. Ney. Improved backing-off for M-gram language modeling. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, volume 1, pages 181-184, 1995.

[18] J. Kunegis, S. Schmidt, A. Lommatzsch, J. Lerner, E. W. De Luca, and S. Albayrak. Spectral analysis of signed graphs for clustering, prediction and visualization. In *Proceedings of the 2010 SIAM International Conference on Data Mining (SDM)*, pages 559-570, 2010.

[19] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419-457, 1995.

[20] L. Page, S. Brin, R. Motwani, and T. Winograd. The PageRank citation ranking: Bringing order to the web. Technical report, Stanford InfoLab, 1998.

[21] Q. Pham, C. Liu, and S. Hoi. DualNet: Continual learning, fast and slow. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.

[22] R. Wu, K. Huang, H. Zhang, Q. Liu, G. Yu, J. Deng, and F. Ye. Information-theoretic dual memory system for continual learning. *Knowledge-Based Systems*, 325:113913, 2025.

[23] R. Eberle. *SCAMPER: Games for Imagination Development*. D.O.K. Publishers, 1971.

[24] A. F. Osborn. *Applied Imagination: Principles and Procedures of Creative Problem-Solving*. Charles Scribner's Sons, 1953 (3rd rev. ed.,

1963).

[25] R. F. Foelix. *Biology of Spiders*, 3rd edition. Oxford University Press, 2011.

[26] R. Hergenröder and F. G. Barth. The release of attack and escape behavior by vibratory stimuli in a wandering spider (*Cupiennius salei* Keys.). *Journal of Comparative Physiology A*, 152:347-358, 1983.

[27] F. Monroe and A. D. Rubin. Keystroke dynamics as a biometric for authentication. *Future Generation Computer Systems*, 16(4):351-359, 2000.

[28] J. A. Russell. A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6):1161-1178, 1980.

[29] J. B. Walther. Computer-mediated communication: Impersonal, interpersonal, and hyperpersonal interaction. *Communication Research*, 23(1):3-43, 1996.

[30] M. Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298-305, 1973.

[31] M. Belkin and P. Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373-1396, 2003.

[32] J. Duprez, M. Stokkermans, L. Drijvers, and M. X. Cohen. Synchronization between keyboard typing and neural oscillations. *Journal of Cognitive Neuroscience*, 33(5):887-901, 2021.

[33] F. Richlan, J. Schubert, R. Mayer, F. Hutzler, and M. Kronbichler. Action video gaming and the brain: fMRI effects without behavioral effects in visual and verbal cognitive tasks. *Brain and Behavior*, 8(1):e00877, 2018.

Appendix A. Correspondence to the Motivating Analogy

The architecture in this paper was first described informally, using a spider-web analogy, before being formalized. This table records that correspondence directly, for reference. It is a pedagogical mapping, not independent evidence for anything: the formal claims stand or fall on the citations and proofs in the sections cited below, not on the analogy that motivated them.

Status note (Revision 2). Two rows below are marked [literal] rather than [pedagogical]. Every other row in this table is a loose, motivating metaphor — nothing forces a personal intent graph to resemble a spider web beyond the fact that thinking of it that way helped generate the design. The two marked rows are different in kind: Section 3.2’s silk-type differentiation and Section 6.2’s frequency-derived valence are built directly from cited, verified findings in arachnology [25] and neuroethology [26] respectively, not from a metaphor loosely inspired by spiders in general. Collapsing that distinction — treating a cited biological mechanism and a pedagogical analogy as the same kind of justification — would overstate what the analogy table as a whole is doing, which is why it is flagged explicitly rather than left for the reader to infer.

Status note (Revision 3). Four rows below (the last four in the table) are added for Section 6.9’s analogy-derived mechanisms, marked [pedagogical] exactly like the rows above rather than [literal, cited] like the two Revision 2 rows: Section 0 of the addendum is explicit that these four mechanisms are analogy used to invent new, untested mechanism, not claims about literal spider behavior beyond the two already-cited findings elsewhere in this table. Two of Section 6.9’s eight subsections deliberately do *not* appear below at all: the two-axis affective framework (6.9.1) and the communication-register prior (6.9.8) derive from established, cited literature (Russell [28]; Walther [29], Section 2.8), not from the spider analogy, and their correspondence is a citation relationship, stated in Section 2.8, rather than a pedagogical mapping of the kind this table records.

Analogy element	Formal object	Section
The web itself, laid over one place [pedagogical]	The node set V and the single, function-typed edge set built on it	3.2
Radial threads carrying vibration from far-off prey [pedagogical]	Diffusion of the seed signal through transmission-type edges, $W_{\text{transmission}}$ (heat kernel / personalized PageRank)	4.1, 4.2
The spider using a nearby strand instead of the exact spot under its feet [pedagogical]	Ranking candidates by resulting diffusion value, not by direct adjacency to the input	4.4
A thread thickening with repeated traffic [pedagogical]	Raw-weight reinforcement (Ant Colony Optimization dynamics)	3.2, Eq. (1)
A depleted patch the spider avoids returning to [pedagogical]	Tabu-style exhaustion suppression of recently-satisfied patterns	2.6, 3.2 Eq. (14)
Dry, stiff radial and frame silk carries signal; sticky, elastic capture-spiral silk traps and holds, and the two are never confused for one another [literal, cited]	Transmission-fidelity-based role assignment: an edge is transmission-type (propagates) or capture-type (does not), by $TF(u, v, t)$ alone	3.2, Eqs. (16)–(17)
Low-frequency substrate vibration (3–4 Hz) drives approach; high-frequency vibration (350–460 Hz) drives withdrawal, read directly off the signal’s physical shape with no semantic interpretation [literal, cited]	Frequency-derived valence proxy $val'(c_t)$, computed from typing cadence and correction rate rather than word content	6.2, Eqs. (7), (20)
The web reorganizing after an unexpected catch [pedagogical]	The valence-blindness problem and the signed, valence-gated update	5, 6
Several threads struggling at once, registered together rather than as isolated strikes [pedagogical]	Compounding cluster reinforcement across the active-prey set $P(t)$	6.9.2, Eqs. (24)–(26)
Engagement and reach growing the longer a struggle continues [pedagogical]	Growing engagement radius: $\alpha_{\text{effective}}$ shrinking as a saturating function of sustained-arousal streak length	6.9.3, Eqs. (27)–(29)
Being hurt at one spot and, separately, growing generally warier for a while afterward [pedagogical]	The shock-retreat event: immediate local suppression plus a temporary, systemic rise in the necessity-gating threshold	6.9.4, Eqs. (30)–(31), (36)
Settling back to baseline once a struggle ends, differently depending on whether prey was actually secured [pedagogical]	The grace-period settling event: confirmed- vs. unconfirmed-streak decay of recent reinforcement and radius growth	6.9.5, Eq. (32)

Status note (Revision 4). No new row is added for Section 4.5. Unlike Revisions 2 and 3, this addendum does not introduce a new element of

the spider-web analogy — it takes the first row above, “the web itself, laid over one place,” and gives one specific, narrow piece of it a literal, computable form for the first time: not the whole spatial gestalt the row represents, only the claim that a node’s connectivity breadth ($\varphi(u)$) predicts its position relative to the web’s center (Section 4.5.1, Eq. (37)). That row remains marked [pedagogical] rather than being upgraded to [literal], because most of what it represents — the general sense of a web “laid over” a place — still has no formal claim attached to it beyond that one narrow, and still unverified, correlation. Marking the whole row [literal] on the strength of one untested hypothesis inside it would overstate exactly the kind of thing this table’s own Revision 2 status note above warns against conflating.

(Retired, Revision 2: “Eggs laid in a safe place → promotion of a sustained edge into the protected stable channel, Section 3.3, Eq. (3).” There is no longer a protected stable channel for an edge to be promoted into; the row above on silk-type role assignment replaces it, and is marked [literal, cited] rather than [pedagogical] for the reason stated above — it is a stronger, differently-grounded claim than the row it replaces, not a renaming of it.)